



# Optimizing supply chain operations using advanced Time-Series Mixer models for demand forecasting and inventory under uncertain demand

Thi Ngoc Anh Nguyen <sup>a,b</sup>, Thi Xuan Hoa Nguyen <sup>b,c,\*</sup>, Ngoc Thang Tran <sup>a,b</sup>,  
Thi Ha Nguyen <sup>a</sup>, Phuong Anh Nguyen <sup>c</sup>, Hai Anh Vu <sup>c</sup>

<sup>a</sup> Faculty of Applied Mathematics and Informatics, Hanoi University of Science and Technology, No. 1, Dai Co Viet Road, Hanoi, 100000, Vietnam

<sup>b</sup> Institute for Digital Technology and Economy, Hanoi University of Science and Technology, No. 1, Dai Co Viet Road, Hanoi, 100000, Vietnam

<sup>c</sup> School of Economics and Management, Hanoi University of Science and Technology, No. 1, Dai Co Viet Road, Hanoi, 100000, Vietnam

## ARTICLE INFO

### Keywords:

Demand forecasting  
Inventory management  
Supply chain management  
Time series  
RevIN-TSMixer

## ABSTRACT

Effective inventory management in the presence of uncertain demand necessitates forecasting models that strike a balance between accuracy and cost efficiency. This study introduces an approach that incorporates Reversible Instance Normalization (RevIN) with Time-Series Mixer (TSMixer), referred to as RevIN-TSMixer, a transformer-based model built on the Multi-Layer Perceptron Mixer architecture, leveraging Multi-Layer Perceptron (MLP) across temporal and feature dimensions. The proposed approach is evaluated in two scenarios: error optimization, aimed at minimizing forecasting errors, and total cost optimization, emphasizing practical cost-effectiveness. The results demonstrate that RevIN-TSMixer consistently outperforms the standard TSMixer model. These advancements are credited to RevIN's ability to address data shifts and improve adaptability to fluctuating demand patterns. To further improve supply chain management, the study integrates a probabilistic inventory model under a continuous review  $(r, q)$  inventory policy with the proposed forecasting model. This integration includes safety stock calculations, reorder point determination, and comprehensive cost analyses, reducing shortages, and optimizing inventory strategies. Validated through experimental scenarios, the proposed framework highlights the synergy between accurate forecasting and adaptive inventory management. By demonstrating robust forecast accuracy and significant cost reductions, RevIN-TSMixer establishes a state-of-the-art solution for supply chain optimization under uncertain demand conditions.

## 1. Introduction

In recent years, inventory management has become a crucial aspect of supply chain management, as inventory costs can account for 20%–40% of a product's total value (Manatkar et al., 2016). Effective inventory management is essential to reduce these costs and improve overall supply chain efficiency, as it relies on accurately quantifying demand uncertainty. This uncertainty is typically addressed through demand forecasts in the lead time (Kourentzes et al., 2020).

Product demand forecasting accuracy is essential for effective supply chain management and corporate planning (Zhuang et al., 2022). Accurate demand forecasts enable companies to optimize inventory levels, reduce holding costs, and improve customer happiness by ensuring access to products (Cui et al., 2024). Conversely, inaccurate forecasts can lead to either shortages or surplus inventory situations, both of which negatively affect operational efficiency and financial profitability

(Cui et al., 2024; Zhuang et al., 2022). Organizations can significantly improve the precision of their demand forecasts by employing sophisticated data analysis and machine learning techniques (Wu et al., 2023).

The correlation between demand forecasting and inventory optimization lies in their mutual dependence on achieving operational efficiency and customer satisfaction. Accurate demand forecasting provides critical input for inventory optimization by estimating future product requirements, allowing businesses to strike a balance between overstocking and understocking (Kück & Freitag, 2021). This minimizes costs associated with excessive storage, spoilage, or obsolescence while simultaneously reducing the risk of stockout that leads to lost sales opportunities and dissatisfied customers. Conversely, inaccurate demand forecasting disrupts inventory optimization efforts. Excess inventory may result in wasted storage resources, higher labor and insurance expenses, and diminished product quality, while stockouts can lead to lost sales, lower customer satisfaction, and reduced brand loyalty

\* Corresponding author.

E-mail addresses: [anh.nguyenthingoc@hust.edu.vn](mailto:anh.nguyenthingoc@hust.edu.vn) (T.N.A. Nguyen), [hoa.nguyenthixuan@hust.edu.vn](mailto:hoa.nguyenthixuan@hust.edu.vn) (T.X.H. Nguyen), [thang.tranngoc@hust.edu.vn](mailto:thang.tranngoc@hust.edu.vn) (N.T. Tran), [jenahanguyen2404@gmail.com](mailto:jenahanguyen2404@gmail.com) (T.H. Nguyen), [anh.np242077m@sis.hust.edu.vn](mailto:anh.np242077m@sis.hust.edu.vn) (P.A. Nguyen), [anh.vuhai@hust.edu.vn](mailto:anh.vuhai@hust.edu.vn) (H.A. Vu).

<https://doi.org/10.1016/j.eswa.2025.128955>

Received 25 March 2025; Received in revised form 30 May 2025; Accepted 6 July 2025

Available online 21 July 2025

0957-4174/© 2025 Published by Elsevier Ltd.

**Table 1**  
List of abbreviations.

| Abbreviation | Definition                      | Abbreviation | Definition                               |
|--------------|---------------------------------|--------------|------------------------------------------|
| TSMixer      | Time-Series Mixer               | RevIN        | Reversible Instance Normalization        |
| LSTM         | Long Short-Term Memory          | FC           | Fully Connected                          |
| GRU          | Gated Recurrent Unit            | ReLU         | Rectified Linear Unit                    |
| CNN          | Convolutional Neural Network    | MSE          | Mean Squared Error                       |
| RNN          | Recurrent Neural Network        | MAE          | Mean Absolute Error                      |
| LightGBM     | Light Gradient Boosting Machine | RMSE         | Root Mean Squared Error                  |
| SVR          | Support Vector Regression       | MAPE         | Mean Absolute Percentage Error           |
| DDI          | Downstream Demand Inference     | EOQ          | Economic Order Quantity                  |
| ES           | Exponential Smoothing           | FDI          | Foreign Direct Investment                |
| SES          | Single Exponential Smoothing    | MMSE         | Minimum Mean Squared Error               |
| WMA          | Weighted Moving Average         | CPI          | Consumer Price Index                     |
| VAR          | Vector Autoregressive           | ARIMA        | Autoregressive Integrated Moving Average |

(Cui et al., 2024). Thus, effective integration of demand forecasting into inventory optimization enables informed decision-making regarding production and procurement quantities, order timing, and the appropriate safety stock level (Syntetos et al., 2009). This ensures a responsive and efficient supply chain, enhancing operational performance and customer experience. In recent years, the term “inventory forecasting” has been used by many researchers to describe the intersection of these two fields (Goltsoy et al., 2022).

However, it is undeniable that the most accurate forecast does not necessarily result in optimal inventory performance (Kourentzes et al., 2020; Tratar, 2010). In other words, the most effective demand forecasting method for minimizing inventory costs is not necessarily always the one with the highest forecast accuracy. Typically, forecasting methods are optimized on the in-sample mean absolute error (MAE), mean squared error (MSE), root mean squared error (RMSE), or mean absolute percentage error (MAPE). These metrics, often employed as cost functions for the parameters optimization, are designed to produce optimal predictions for the average demand pattern (Gneiting, 2011). However, forecasting for inventory management should not just focus on optimizing the mean or median value of demand. These measures do not consider the asymmetric costs of over- and under-stocking and assumptions about demand distribution, which can lead to ineffective inventory performance (Kourentzes et al., 2020).

To address this challenge, we propose optimizing forecasts directly based on decision variables rather than traditional divergence metrics, such as MSE, MAE, RMSE, or MAPE. Instead, we integrate the inventory optimization model into the forecasting model using inventory performance metrics as primary optimization criteria. In particular, we integrated a deep learning-based demand forecasting methodology, TSMixer, with a probabilistic inventory model. The study examines the RevIN approach to improve the performance of the TSMixer model and demonstrates its superiority in inventory optimization when combined with RevIN.

This paper makes several key contributions to the field of demand forecasting and inventory management as follows:

- This study provides a comprehensive analysis of multivariate time series forecasting problems, focusing on their implications for inventory management. It explores various forecasting models, offers insights into hyperparameter selection methods, and investigates strategies for optimizing inventory performance.
- This study introduces the integration of the RevIN approach into the standard TSMixer model, forming the RevIN-TSMixer model to enhance forecasting accuracy and inventory performance. This integration enables the TSMixer model to handle non-stationary demand patterns more effectively, reducing the impact of seasonality and fluctuations in data distribution.
- This study employed a probabilistic inventory model for uncertain demand that considers shortage cost as a variable driving other key inventory metrics. This approach enables a thorough evaluation of

how changes in shortage cost influence stockout probability, safety stock, reorder point, and total costs, providing valuable insights into the relationship between forecast accuracy and inventory performance.

- This study evaluates the proposed model through two experimental scenarios: (1) error optimization, which considers forecasting and inventory optimization as two separate phases, and (2) cost optimization, which integrates the inventory model into the forecasting model by employing total inventory cost as an objective function for the forecasting model. Then, an in-depth discussion on model robustness is presented, highlighting the effectiveness of RevIN-TSMixer in balancing forecasting accuracy and cost efficiency in inventory management.

These contributions collectively advance both the theoretical and practical aspects of intelligent supply chain management. The paper is structured as follows: In [Section 1](#), a thorough summary of the research is provided, outlining the main issue addressed within the framework of product demand forecasting for optimizing inventory. [Section 2](#) shows the related works about demand forecasting, and [Section 3](#) introduces the Reversible Instance Normalization TSMixer model for predicting the behavior of several time series and policies with optimal order quantity in inventory management. [Section 4](#) specifically addresses the practical use and outcomes of experiments carried out on real datasets. [Section 6](#) describes the conclusion and discussion. The abbreviations used in this article are listed in [Tables 1](#) and [2](#).

## 2. Related works

Demand forecasting is a critical component of supply chain management, with its accuracy directly impacting production planning, inventory management, and distribution processes (Douaioui et al., 2024). Early research efforts primarily focused on improving forecasting accuracy, often without considering the operational cost implications. Traditional statistical models such as ARIMA and Exponential Smoothing were widely used due to their robustness in capturing temporal patterns. To address the limitations of traditional modeling, recent approaches have brought to light the need to focus on machine learning and deep learning. To increase the accuracy, Owais (2025) proposed a hybrid simulation and deep learning approach for travel demand forecasting. In addition, Wu et al. (2024) developed a knowledge-based dual-tuning model (KbBcM) using LSTM and penalty attention to minimize the consequences in urban water demand forecasting, selecting superior performance through novel evaluation metrics such as average expected trend efficiency. In addition to numerical data, Zhang et al. (2025) used textual data by extracting information from online reviews such as the BERT model and Transformer-based architecture to transform into embedding vectors. When adding more data, the model accuracy increased significantly compared to the matching model. Riachy et al. (2025) introduced feature engineering techniques that integrate con-

**Table 2**  
Notations related to the proposed model.

| Symbol                         | Definition                                                            | Symbol                                     | Definition                                                                                                                       |
|--------------------------------|-----------------------------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameters</b>              |                                                                       |                                            |                                                                                                                                  |
| $T$                            | Length of the input sequence                                          | $N$                                        | Length of the forecasted sequence                                                                                                |
| $K$                            | Number of mixer layers in model                                       | $p$                                        | Dropout rate                                                                                                                     |
| $M$                            | Batch size in model                                                   | $\text{ff\_dim}$                           | The number of units in the layer                                                                                                 |
| $\text{lr}$                    | The learning rate of the model                                        |                                            |                                                                                                                                  |
| <b>Variables</b>               |                                                                       |                                            |                                                                                                                                  |
| $\mathcal{X}$                  | Historical time series                                                | $\hat{\mathcal{X}}$                        | Forecasted time series                                                                                                           |
| $S$                            | The length of the input data                                          | $C$                                        | The number of features in the input dataset                                                                                      |
| $F$                            | The number of features in the predicted dataset ( $F \leq C$ )        | $\mathbf{X}_t$                             | An observation of the input dataset at time $t$                                                                                  |
| $Q$                            | Order quantity                                                        | $Q^*$                                      | Optimal order quantity                                                                                                           |
| $\mu_D$                        | Average monthly demand                                                | $\sigma_D$                                 | Standard deviation of demand                                                                                                     |
| $D_i$                          | The actual demand for the $i$ -th month                               | $F_i$                                      | The forecasted demand for the $i$ -th month                                                                                      |
| $SS$                           | Safety stock                                                          | $r$                                        | Reorder point                                                                                                                    |
| $\alpha$                       | Service level                                                         | $z$                                        | z-score corresponding to service level                                                                                           |
| $c_s$                          | Shortage cost per unit                                                | $E(c_s)$                                   | Expected shortage cost                                                                                                           |
| $\mathcal{L}(z)$               | Loss function                                                         | $c_h$                                      | Total holding cost                                                                                                               |
| $c_o$                          | Total ordering cost                                                   | $TC$                                       | Total cost                                                                                                                       |
| <b>Constants</b>               |                                                                       |                                            |                                                                                                                                  |
| $h$                            | Holding cost per unit (\$ / unit / month)                             | $o$                                        | Ordering cost per order (\$ / order)                                                                                             |
| $L$                            | Lead time (month)                                                     | $P$                                        | Price (\$)                                                                                                                       |
| <b>Tensors</b>                 |                                                                       |                                            |                                                                                                                                  |
| $\mathcal{X}^{(d)}$            | The input tensor of the proposed model                                | $\hat{\mathbf{Y}}^{(d)}$                   | The output tensor of the proposed model                                                                                          |
| $\mathcal{B}^{(d)}$            | The input tensor of TSMixer model                                     | $\mathcal{B}^{(d)}_{norm}$                 | The tensor after 2D Batch Normalization in the Time-Mixing Layer                                                                 |
| $\mathcal{B}^{(d)}_{t,trans1}$ | The tensor after the first Tensor Transpose in the Time-Mixing Layer  | $\mathcal{B}^{(d)}_{t,mlp}$                | The tensor after MLP <sub>time</sub> in the Time-Mixing Layer                                                                    |
| $\mathcal{B}^{(d)}_{t,trans2}$ | The tensor after the second Tensor Transpose in the Time-Mixing Layer | $\mathcal{B}^{(d)}_{time}$                 | The output tensor of the Time-Mixing Layer                                                                                       |
| $\mathcal{B}^{(d)}_{f,norm}$   | The tensor after 2D Batch Normalization in the Feature-Mixing Layer   | $\mathcal{B}^{(d)}_{f,mlp}$                | The tensor after MLP <sub>feature</sub> in the Feature-Mixing Layer                                                              |
| $\mathcal{U}^{(d)}_{f,mlp}$    | The intermediate tensor of MLP <sub>feature</sub>                     | $\mathcal{B}^{(d)}_{feature}$              | The output tensor of the Feature-Mixing Layer                                                                                    |
| $\mathcal{B}^{(d)}_{mixing}$   | The output tensor after $K$ -th of the Mixing Layer                   | $\mathcal{B}^{(d)}_{p,trans1}$             | The tensor after the first Tensor Transpose in the Temporal Projection Layer                                                     |
| $\mathcal{B}^{(d)}_{p,fc}$     | The tensor after the Fully Connected in the Temporal Projection Layer | $\hat{\mathcal{B}}^{(d)}$                  | The tensor after the second Tensor Transpose in the Temporal Projection Layer, which is also the output tensor of TSMixer model. |
| <b>Matrices and Vectors</b>    |                                                                       |                                            |                                                                                                                                  |
| $\mathbf{W}_1$                 | The weight matrix of Time-Mixing Layer                                | $\mathbf{W}_2, \mathbf{W}_3, \mathbf{W}_4$ | The weight matrices of Feature-Mixing Layer                                                                                      |
| $\mathbf{W}_5$                 | The weight matrix of Temporal Projection Layer                        | $\mathbf{b}_i$                             | The bias vectors corresponding to the weight matrix $\mathbf{W}_i$ with $i = 1, 2, \dots, 5$                                     |
| <b>Operations</b>              |                                                                       |                                            |                                                                                                                                  |
| $\oplus$                       | The element-wise addition with the bias vector                        | $*$                                        | The matrix multiplication between tensor and weight matrix                                                                       |

texts to improve post-pandemic retail demand forecasting using deep learning models. Furthermore, Cui et al. (2024) used LSTM attention, which experimentally demonstrated good accuracy and robustness in the supply chain. Demand forecasting studies, although in various fields, the combination of deep learning models and new innovative techniques has improved the forecasting accuracy in the context of complex and uncertain environments. Despite these advancements, many methods overlook the interplay between demand forecasting and inventory optimization, treating them as separate processes (Kourentzes et al., 2020).

In response, many researchers introduced hybrid models that integrated forecasting with inventory management to consider both predictive performance and downstream cost implications jointly. In the past, many studies have shown that higher forecast accuracy reduces safety stock requirements, minimizing holding costs while maintaining desired service levels. Researchers have highlighted how forecast accuracy directly influences critical inventory parameters, such as average inventory levels (Tliche et al., 2020), inventory costs (Ma, 2024; Seyedan et al., 2023), service level (Babai et al., 2022; Hasni et al., 2024; Wellens et al., 2024), stockout rates (Demizu et al., 2023), replenishment cycle (Salas-Navarro et al., 2023). However, those studies treat forecasting and inventory management as two separate phases. As a result, inventory performance is evaluated based on the outcomes of forecasting models, which often leads to suboptimal solutions for inventory optimization.

More recently, research has shifted toward forecasting models that explicitly optimize the trade-off between accuracy and cost. Rather than focusing solely on minimizing forecast error, these models recognize that marginal improvements in accuracy may not always be worth the extra effort. In some cases, trying to improve accuracy too much can lead to higher costs or excessive inventory (Kourentzes et al., 2020). These studies suggest that the most effective method for reducing inventory costs may not necessarily be the one that achieves the highest level of forecast accuracy. Researchers have then employed inventory metrics as evaluation metrics instead of traditional fitting error metrics in forecasting models. These inventory metrics are closely associated with inventory optimization models, e.g., service level, inventory level, and total cost. The common advantage of this method is that it reduces forecast accuracy while improving alignment with inventory performance objectives. For instance, Kourentzes et al. (2020) integrated the service level metric into their forecasting models, reporting a 9% reduction in forecasting accuracy but a significant improvement in inventory performance. van der Haar et al. (2024) used supervised learning to predict optimal order sizes directly rather than first predicting demand, achieving near-optimal results when demand is stationary. Other studies, such as Wellens et al. (2024) and Wang et al. (2024), employed ML and probabilistic methods that explicitly incorporate inventory evaluation metrics – like holding and shortage costs – into the model selection or training process. It results in an improvement in forecast accuracy. Table 3 pro-

**Table 3**  
Related work on the integration of forecasting and inventory optimization.

| Authors                     | Forecast                                |                 |                                                 | Inventory                    |                                    | Integration level |       |
|-----------------------------|-----------------------------------------|-----------------|-------------------------------------------------|------------------------------|------------------------------------|-------------------|-------|
|                             | Methods                                 | Response        | Feature                                         | Models                       | Metrics                            | Sequential        | Joint |
| Kourentzes et al. (2020)    | Exponential Smoothing                   | Demand quantity | Historical sales/demand                         | OUT policy                   | Service level                      |                   | ✓     |
| Tliche et al. (2020)        | Moving Averages approach                | Demand quantity | Historical demand                               | OUT policy                   | Average inventory level            | ✓                 |       |
| Babai et al. (2022)         | Simple Exponential Smoothing            | Demand quantity | Historical demand                               | OUT policy                   | Service level, cost                | ✓                 |       |
| Seyedan et al. (2023)       | Ensemble model (MLP, LSTM, CNN)         | Demand quantity | Historical demand, price, holidays              | OUT policy                   | Cost                               | ✓                 |       |
| Demizu et al. (2023)        | Bayesian Neural Networks with MA        | Demand quantity | Historical sales, product specs, store location | Model-based DRL              | Inventory turnover, stockout rates | ✓                 |       |
| Salas-Navarro et al. (2023) | Probabilistic demand model              | Demand quantity | Historical sales/demand                         | EPQ                          | Replenishment cycle, order times   |                   | ✓     |
| Hasni et al. (2024)         | LSTM-SVR                                | Demand quantity | Historical sales/demand                         | Simulation                   | Service level, inventory level     | ✓                 |       |
| Sukolkit et al. (2024)      | Ensemble model (Prophet, SVR, CNN, RNN) | Demand quantity | Promotions, sales trends                        | (r,Q) policy with EOQ        | Cost                               | ✓                 |       |
| van der Haar et al. (2024)  | Supervised learning + LightGBM          | Demand quantity | Promotions, sales trends                        | Quantile-based replenishment | Cost                               |                   | ✓     |
| Ma (2024)                   | ASG-RNN                                 | Demand quantity | Historical sales/demand                         | Quantile-based replenishment | Replenishment cost                 | ✓                 |       |
| Wellens et al. (2024)       | Tree-based ML                           | Demand quantity | Price, promotions, holidays                     | OUT policy                   | Service level, cost                |                   | ✓     |
| Wang et al. (2024)          | Probabilistic combinations              | Demand quantity | Historical sales/demand                         | OUT policy                   | Service level, inventory level     |                   | ✓     |
| This study (2025)           | RevIN-TSMixer                           | Demand quantity | Historical demand, macroeconomic                | (r,Q) policy with EOQ        | Shortage cost                      |                   | ✓     |

vides a comprehensive comparison of our study with existing research, detailing model architectures, input features, response variables, and performance metrics.

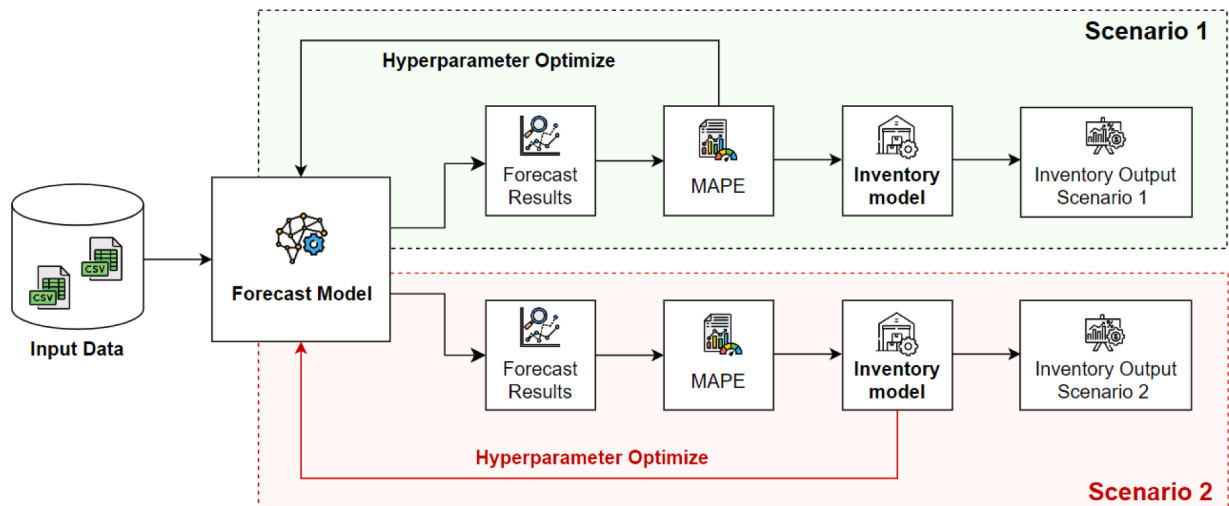
Despite extensive research in time series forecasting, inventory management, and their correlation, several gaps remain unaddressed. First, most studies either prioritize forecast accuracy or focus on minimizing inventory costs, with limited efforts to align these goals dynamically. In particular, inventory costs are often considered secondary in forecasting evaluations or treated separately in a post-forecasting phase. Second, while existing forecasting models achieve high accuracy, they often lack mechanisms to optimize inventory costs effectively. As a result, these models may improve short-term accuracy but fail to deliver tangible cost benefits, such as reducing stockouts, minimizing holding costs, or optimizing reorder points. Lastly, existing studies optimize demand forecast results without considering the role of shortage cost in shaping inventory decisions. Since shortage costs directly influence reorder point, safety stock, and overall inventory performance, failing to account for them can lead to suboptimal inventory strategies.

### 3. Methodology

#### 3.1. Overview

To thoroughly assess the interaction between demand forecasting accuracy and inventory management efficiency, we propose two experimental scenarios, each addressing complementary aspects of the problem. These scenarios are designed to evaluate the performance of forecasting models and their subsequent impact on inventory decisions, enabling a comprehensive analysis of the proposed methods. Proposed methodology with two scenarios is presented in Fig. 1.

Scenario 1 focuses on isolating the performance of forecasting models to evaluate their contribution to inventory management. The process begins with data collection, combining internal company data and external crawled data, followed by preprocessing. Two forecasting strategies are employed: a standalone TSMixer model and a hybrid RevIN-TSMixer model. Forecast accuracy is evaluated using metrics such as MSE, MAE, RMSE, and MAPE. Among these, MAPE is the primary metric for ac-



**Fig. 1.** Overview of the proposed methodology with two scenarios.

curacy evaluation, as it provides an intuitive measure and facilitates comparative analysis across datasets. The resulting forecasts, combined with historical sales and inventory data, are fed into an inventory model based on the  $(r, q)$  policy, where  $r$  represents the reorder point and  $q$  denotes the order quantity. These inputs are used to compute critical inventory outputs, including safety stock, reorder points, and total costs. This scenario provides insights into how forecasting techniques impact inventory management when evaluated independently.

Scenario 2 integrates inventory management outcomes into the evaluation process, emphasizing an end-to-end perspective. The workflow mirrors Scenario 1, encompassing data collection, preprocessing, and forecasting using the same models. However, this scenario enhances the hyperparameter optimization process by introducing the Total Cost metric as the primary objective, ensuring that the optimization accounts for the financial implications of forecasting errors such as overstocking or stockouts. Given that shortage levels fluctuate, the evaluation focuses on minimizing the Total Cost to adapt to varying demand conditions. MAPE is a supplementary metric to provide additional insights into forecast accuracy. The inventory model employs the  $(r, q)$  policy, where  $r$  represents the reorder point and  $q$  denotes the order quantity to generate inventory decisions. This integrated evaluation framework highlights the practical and economic implications of forecasting accuracy, ensuring that the models are optimized for both prediction reliability and cost-effectiveness in dynamic inventory management scenarios.

By comparing these two scenarios, we aim to balance model complexity, forecasting performance, and inventory efficiency, ensuring that the proposed methods are both robust and cost-effective for real-world applications.

### 3.2. Demand forecasting

#### 3.2.1. Forecasting problem formulation

Due to practical requirements in inventory management, this study proposes forecasting product demand with a lead time of three months. This three-month forecasting cycle allows businesses sufficient time to adjust their supply and production plans. In practice, ordering, inventory adjustments, and production planning decisions are often based on demand forecasts for the next three months. Forecasting demand in three-month cycles enhances the accuracy of short-term forecasts and enables businesses to proactively prepare inventory, avoiding shortages and excess stock. Furthermore, the three-month time frame aligns well with the production and supply cycles in many industries, helping to optimize processes and mitigate risks within the supply chain.

To address the forecasting requirements, we formulate the problem as follows: the objective is to predict the future time series, denoted as  $\hat{\mathcal{X}}$ , based on the historical time series  $\mathcal{X}$ . The historical time series is defined as  $\mathcal{X} = \{\mathbf{X}_t\}_{t=1}^S \in \mathbb{R}^{S \times C}$ , where  $\mathbf{X}_t = \{X_{t,1}, X_{t,2}, \dots, X_{t,C}\}$  represents an observation of the input dataset at time  $t$ . Here,  $S \in \mathbb{N}^*$  is the length of the input dataset, and  $C \in \mathbb{N}^*$  is the number of features in the input data. The predicted time series  $\hat{\mathcal{X}}$ , is represented as  $\hat{\mathcal{X}} = \{\hat{\mathbf{X}}_t\}_{t=S+4}^{S+N+3} \in \mathbb{R}^{N \times F}$ , where  $\hat{\mathbf{X}}_t = \{\hat{X}_{t,1}, \hat{X}_{t,2}, \dots, \hat{X}_{t,F}\}$  corresponds to the predicted observation at time  $t$ . In this formulation,  $N \in \mathbb{N}^*$  denotes the number of forecast steps, and  $F \in \mathbb{N}^*$  represents either the number of features in the predicted time series or the features of the prediction dataset, depending on the context. The input time series  $\mathcal{X}$  and the predicted time series  $\hat{\mathcal{X}}$  is formally described in Eq. (1).

$$\mathcal{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_S \end{bmatrix} = \begin{bmatrix} X_{1,1} & X_{1,2} & \dots & X_{1,C} \\ X_{2,1} & X_{2,2} & \dots & X_{2,C} \\ \vdots & \vdots & \ddots & \vdots \\ X_{S,1} & X_{S,2} & \dots & X_{S,C} \end{bmatrix},$$

$$\hat{\mathcal{X}} = \begin{bmatrix} \hat{\mathbf{X}}_{S+4} \\ \hat{\mathbf{X}}_{S+5} \\ \vdots \\ \hat{\mathbf{X}}_{S+N+3} \end{bmatrix} = \begin{bmatrix} \hat{X}_{S+4,1} & \hat{X}_{S+4,2} & \dots & \hat{X}_{S+4,F} \\ \hat{X}_{S+5,1} & \hat{X}_{S+5,2} & \dots & \hat{X}_{S+5,F} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{X}_{S+N+3,1} & \hat{X}_{S+N+3,2} & \dots & \hat{X}_{S+N+3,F} \end{bmatrix}. \quad (1)$$

#### 3.2.2. TSMixer model

The Time-Series Mixer (TSMixer) model represents a novel time series forecasting paradigm designed to handle the intricate dynamics of real-world multivariate data. Unlike traditional recurrent or attention-based architectures, TSMixer leverages a straightforward yet powerful MLP-based design, offering a modular structure that aligns well with the demands of financial modeling. By mixing operations across both temporal and feature dimensions, TSMixer captures trends over time and cross-sectional interactions between variables, making it particularly effective in applications like stock realized volatility forecasting. This dual-focus approach enhances its ability to process complex, multivariate time series data, a critical requirement in financial forecasting tasks.

The TSMixer model architecture comprises two main components: the Mixing Layer and the Temporal Projection Layer. The Mixing Layer consists of two sub-layers, namely the Temporal Mixing Layer and the Feature Mixing Layer, which operate sequentially to extract temporal and feature-specific patterns. These layers process tensors derived from the input dataset, transforming raw historical time series data into meaningful representations for forecasting.

Let us define the input as a tensor  $\mathcal{B}^{(d)} \in \mathbb{R}^{M \times T \times C}$ , where  $M$  represents the number of samples (or batches),  $T$  denotes the number of time steps, and  $C$  is the number of input features.

##### Time-Mixing Layer

The Time-Mixing Layer (*Time-Mixing*) performs transformations along the time dimension to capture temporal dependencies in the input data, which is crucial for modeling time-series or sequential data. This layer is designed to effectively model the dynamic relationships between data points at different time steps, allowing the model to better understand the underlying patterns and trends. The architecture of the Time-Mixing Layer, with its detailed configuration and operation, is depicted in Fig. 2.

The process begins with a 2D Batch Normalization applied to the input tensor  $\mathcal{B}^{(d)} \in \mathbb{R}^{M \times T \times C}$ , yielding a normalized tensor  $\mathcal{B}_{t,norm}^{(d)} \in \mathbb{R}^{M \times T \times C}$ . This step ensures that each feature is normalized across the batch, improving model convergence and stability. Specifically, the normalization is performed along the  $M \times C$  dimensions, which correspond to the batch size and feature dimension, respectively. Normalizing the features in each time step  $T$  mitigates fluctuations in the feature distribution, leading to faster convergence and better generalization. The normalization is expressed in Eq. (2).

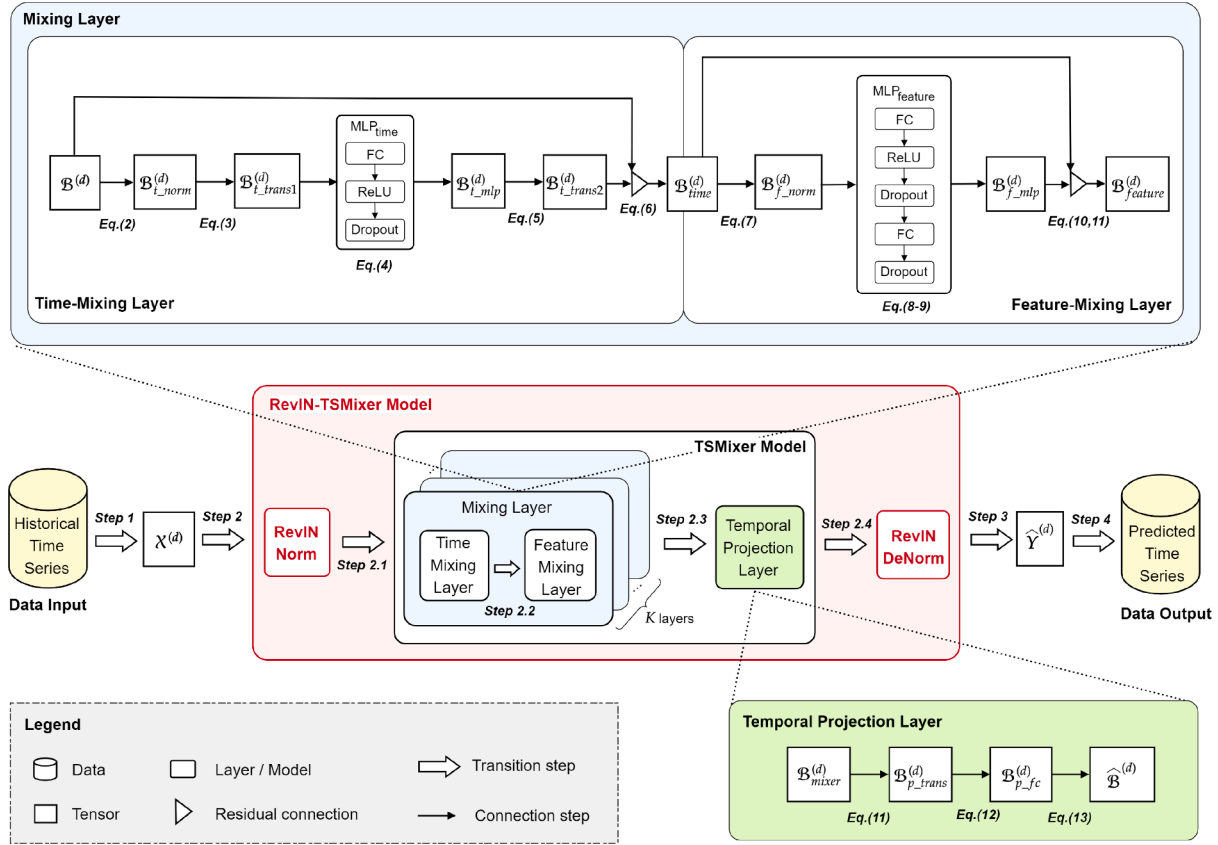
$$\mathcal{B}_{t,norm}^{(d)} = 2DBatchNorm(\mathcal{B}^{(d)}). \quad (2)$$

To facilitate temporal mixing, the tensor is transposed to align the time steps for processing by the  $MLP_{time}$  layer. The time dimension is rearranged by transposing the input tensor to enable the model to learn temporal dependencies more effectively. This transformation ensures that each time step is treated as a separate entity, allowing the  $MLP_{time}$  layer to capture the relationships across different time steps in the sequence. The transposed tensor,  $\mathcal{B}_{t,trans1}^{(d)}$ , is defined in Eq. (3).

$$\mathcal{B}_{t,trans1}^{(d)} = \text{Tensor\_Transpose}(\mathcal{B}_{t,norm}^{(d)}) \in \mathbb{R}^{M \times T \times N}. \quad (3)$$

Next,  $MLP_{time}$  is directly applied to the time dimension. This allows the model to perform temporal mixing by capturing complex relationships between the different time steps. The transformation in  $MLP_{time}$  consists of a linear operation that projects the input features into a higher-dimensional space, followed by an activation function (ReLU) to introduce non-linearity, which enhances the model's ability to learn intricate temporal patterns. Dropout is applied to prevent overfitting by randomly setting some of the neurons to zero during training, and bias is added to account for any shifts in the data. The complete transformation is expressed in Eq. (4), which formalizes the operations performed by the  $MLP_{time}$  layer.

$$\mathcal{B}_{t,mlp}^{(d)} = \text{Dropout}\left(\text{ReLU}\left(\mathcal{B}_{t,trans1}^{(d)} * \mathbf{W}_1 \oplus \mathbf{b}_1\right) p\right), \quad (4)$$



**Fig. 2.** Overall architecture of the proposed model ( Step 1: divide the input sequence into tensors → Step 2: forecasting model: the RevIN-TSMixer model includes transition steps 2.1 to 2.4, while the TSMixer model includes only steps 2.3 and 2.4 → Step 3: output tensor generated after forecasting → Step 4: combine the tensors to form the final output prediction).

where  $\mathbf{W}_1 \in \mathbb{R}^{C \times C}$  is the weight matrix for feature transformation along the time dimension,  $\mathbf{b}_1 \in \mathbb{R}^{1 \times C}$  is the bias vector applied row-wise,  $\text{ReLU}()$  is the activation function, and  $\text{Dropout}()$  is applied with a rate of  $p$ .

After the transformation, the tensor is transposed back to its original dimensions  $M \times T \times C$ , which is defined in Eq. (5).

$$\mathcal{B}_{t\_trans2}^{(d)} = \text{Tensor\_Transpose}(\mathcal{B}_{t\_mlp}^{(d)}) \in \mathbb{R}^{M \times T \times C}. \quad (5)$$

Finally, a residual connection is added by combining the original input tensor  $\mathcal{B}^{(d)}$  with the processed tensor. This step preserves the original features and enhances gradient flow during training. The final output of the Time-Mixing Layer is expressed in Eq. (6).

$$\mathcal{B}_{time}^{(d)} = \mathcal{B}_{t\_trans2}^{(d)} + \mathcal{B}^{(d)} \in \mathbb{R}^{M \times T \times C}. \quad (6)$$

### Feature-Mixing Layer

The Feature-Mixing Layer is designed to capture cross-variable dependencies in the input data, enabling the model to learn complex relationships between features at each time step. This layer transforms the feature space by applying normalization and multi-layer perceptron (MLP) transformations, followed by residual connections, to facilitate efficient learning. The architecture of the Feature-Mixing Layer is depicted in Fig. 2.

The input to the Feature-Mixing Layer is the tensor  $\mathcal{B}_{time}^{(d)} \in \mathbb{R}^{M \times T \times C}$ , which is the output of the Time-Mixing Layer. The tensor has dimensions corresponding to  $M$  samples,  $T$  time steps, and  $C$  features per time step. The first operation is 2D Batch Normalization, which is applied to normalize the features across time and across samples to improve training stability and convergence speed. The normalized tensor is denoted as

$$\mathcal{B}_{f\_norm}^{(d)} \in \mathbb{R}^{M \times T \times C}.$$

$$\mathcal{B}_{f\_norm}^{(d)} = \text{2DBatchNorm}(\mathcal{B}_{time}^{(d)}). \quad (7)$$

Once the input is normalized, we apply a feature transformation using a Multi-Layer Perceptron (MLP) model, which helps capture higher-order feature interactions. The resulting tensor is  $\mathcal{B}_{f\_mlp}^{(d)} \in \mathbb{R}^{M \times T \times F}$ , where  $F$  denotes the number of output features after the transformation. The feature transformation within the  $\text{MLP}_{feature}$  is expressed in Eq. (8).

$$\begin{aligned} \mathcal{U}_{f\_mlp}^{(d)} &= \text{Dropout}(\text{ReLU}(\mathcal{B}_{f\_norm}^{(d)} * \mathbf{W}_2 \oplus \mathbf{b}_2), p), \\ \mathcal{B}_{f\_mlp}^{(d)} &= \text{Dropout}(\mathcal{U}_{f\_mlp}^{(d)} * \mathbf{W}_3 \oplus \mathbf{b}_3, p), \end{aligned} \quad (8)$$

where  $\mathbf{W}_2, \mathbf{W}_3 \in \mathbb{R}^{C \times F}$  the weight matrices for the MLP layers,  $\mathbf{b}_2, \mathbf{b}_3 \in \mathbb{R}^{1 \times F}$  are the bias vectors,  $\text{ReLU}()$  is the activation function, and  $\text{Dropout}()$  is applied with a rate of  $p$ .

The use of a residual connection in the Feature-Mixing Layer helps to mitigate the vanishing gradient problem. It enables the network to learn identity mapping more easily, which is especially useful when the network depth increases. In the case of multivariate time series forecasting (described in Section 3.2.1), the goal is to predict a time series with forecast length  $H$  and the number of predicted features  $F$ . The residual connection handling differs depending on the relationship between  $F$  and  $C$ .

If  $F = C$ , the residual connection is simply the element-wise sum of the normalized features and the input:

$$\mathcal{B}_{feature}^{(d)} = \mathcal{B}_{f\_mlp}^{(d)} + \mathcal{B}_{time}^{(d)} \in \mathbb{R}^{M \times T \times F}. \quad (9)$$

If  $F < C$ , a linear transformation is applied to the residual to match the feature dimensions:

$$\mathbf{B}_{feature}^{(d)} = \left( \mathbf{B}_{f\_mlp}^{(d)} * \mathbf{W}_4 \oplus \mathbf{b}_4 \right) + \mathbf{B}_{time}^{(d)} \in \mathbb{R}^{M \times T \times F}, \quad (10)$$

where  $\mathbf{W}_C \in \mathbb{R}^{C \times F}$  is the weight matrix for the linear transformation, and  $\mathbf{b}_C \in \mathbb{R}^{1 \times F}$  is the bias vector corresponding to the linear transformation.

#### Temporal Projection Layer

The Temporal Projection Layer projects the input data from the original sequence length to the forecast length. This transformation is crucial for aligning the data representation with the target prediction horizon, enabling the model to forecast beyond the current time step effectively. The architecture of the Temporal Projection Layer is shown in Fig. 2.

Given an input tensor  $\mathbf{B}_{mixing}^{(d)} \in \mathbb{R}^{M \times T \times F}$ , which is obtained after the Mixer Layer, the Temporal Projection Layer operates by transforming the columns of the input tensor to project the sequence length from  $T$  to the forecast length  $N$ . The tensor has  $M$  samples,  $T$  time steps, and  $F$  features per time step.

The first step in the process is to transpose the input tensor, which changes the dimensions from  $\mathbb{R}^{M \times T \times F}$  to  $\mathbb{R}^{M \times F \times N}$ :

$$\mathbf{B}_{p\_trans}^{(d)} = \text{Tensor\_Transpose}(\mathbf{B}_{mixing}^{(d)}) \in \mathbb{R}^{M \times F \times N}. \quad (11)$$

This operation reorders the tensor to prepare it for the next transformation. The new tensor has  $M$  samples,  $F$  features, and  $N$  forecast steps.

Next, a Fully Connected (FC) layer is applied in the Temporal Projection Layer. This layer projects the tensor from the input length  $T$  to the forecast length  $N$ , performing a linear transformation on the features. The transformation is given by Eq. (12).

$$\mathbf{B}_{p\_fc}^{(d)} = \mathbf{B}_{p\_trans}^{(d)} * \mathbf{W}_4 \oplus \mathbf{b}_4 \in \mathbb{R}^{M \times F \times N}, \quad (12)$$

where  $\mathbf{W}_4 \in \mathbb{R}^{N \times H}$  is the weight matrix that performs the linear projection from  $F$  features to  $N$  forecasted values, and  $\mathbf{b}_4 \in \mathbb{R}^{1 \times H}$  is the corresponding bias vector.

Finally, a second transpose operation is performed to return the tensor to the original format, but with the forecast length  $H$  as the new time dimension. The result is a tensor of size  $\mathbb{R}^{M \times H \times F}$ :

$$\hat{\mathbf{B}}^{(d)} = \text{Tensor\_Transpose}(\mathbf{B}_{p\_fc}^{(d)}) \in \mathbb{R}^{M \times H \times F}. \quad (13)$$

Thus, the TSMixer model comprises a series of Mixer Layers and a Temporal Projection Layer, which operate on the input tensor extracted from the dataset. These layers work in sequence to transform and project the input data into the desired forecast length. The Mixer Layers are responsible for feature mixing and capturing dependencies across variables, while the Temporal Projection Layer projects the data from the original sequence length to the forecast length. This combination allows the TSMixer model to effectively handle multivariate time series forecasting tasks.

The TSMixer model algorithm is presented in Algorithm 1, which outlines the sequence of operations involved in processing the input tensor and obtaining the forecasted output tensor.

#### 3.2.3. RevIN method

Time series forecasting often faces challenges due to the dynamic nature of time series data, where statistical properties such as the mean and variance can vary over time. This characteristic, known as distribution shift, leads to discrepancies between the train and test datasets, hindering the model's generalization performance. Additionally, input sequences within the dataset may follow different base distributions, leading to inconsistencies among features that can further degrade model performance.

The Reversible Instance Normalization (RevIN) method was proposed to address these issues. RevIN is a normalization and inverse normalization technique specifically designed for time series forecasting tasks to mitigate the effects of distribution shift (Liu et al., 2023). The

#### Algorithm 1: TSMixer Model Algorithm.

**Input:** Input tensor  $\mathbf{B}^{(d)}$ ;  $K$ , the number of Mixing Layers used in the TSMixer model.

**Output:** Predicted output tensor  $\hat{\mathbf{B}}^{(d)}$ .

```

1 Function TSMixer( $\mathbf{B}^{(d)}$ ,  $K$ ):
2   Initialize  $\mathbf{B}_1^{(d)} = \mathbf{B}^{(d)}$ ; // Initialization of the input
   tensor.
3   for  $k = 1$  to  $K$  do
4      $\mathbf{B}_{time}^{(d)} = \text{Time\_Mixing}(\mathbf{B}^{(d)}_k)$ ; // Apply the
     Time-Mixing Layer.
5      $\mathbf{B}_{feature}^{(d)} = \text{Feature\_Mixing}(\mathbf{B}_{time}^{(d)})$ ; // Apply the
     Feature-Mixing Layer.
6     Update  $\mathbf{B}^{(d)}_{k+1} = \mathbf{B}_{feature}^{(d)}$ ; // Pass the result to
     the next layer.
7   Set  $\mathbf{B}_{mixing}^{(d)} = \mathbf{B}^{(d)}_K$ ; // Final tensor after  $K$  Mixing
   Layers.
8    $\hat{\mathbf{B}}^{(d)} = \text{Projection}(\mathbf{B}_{mixing}^{(d)})$ ; // Apply the Temporal
   Projection Layer.
9   return  $\hat{\mathbf{B}}^{(d)}$ ;

```

normalization process stabilizes the input data by removing unstable information such as mean and variance, while the inverse normalization process recovers the original data distribution in the model's outputs, ensuring interpretability and consistency.

The detailed algorithm for the normalization process is presented in Algorithm 2.

#### Algorithm 2: RevIN Normalization Algorithm.

**Input:** Tensor  $\mathcal{X}^{(d)}$  containing  $M$  matrices  
 $\mathbf{X}^{(i)} = [X_{i,j,c}] \in \mathbb{R}^{T \times C}$ , where  $i = 1, 2, \dots, M$ ;  
 $\gamma_c, \beta_c$  are the learnable parameters for feature  $c$ ;  
 $\epsilon$  is a small constant, typically set to  $1e - 5$ .

**Output:** Tensor  $\mathcal{B}^{(d)}$  containing  $M$  matrices  
 $\mathbf{B}^{(i)} = [B_{i,j,c}] \in \mathbb{R}^{T \times C}$ , where  $i = 1, 2, \dots, M$ .

```

1 Function RevIN_Norm( $\mathcal{X}^{(d)}$ ):
2   for  $i = 1$  to  $M$  do
3     for  $c = 1$  to  $C$  do
4       Compute the mean:  $\mu_{i,c} = \frac{1}{T} \sum_{j=1}^T X_{i,j,c}$ ;
5       Compute the variance:  $\sigma_{i,c}^2 = \frac{1}{T} \sum_{j=1}^T (X_{i,j,c} - \mu_{i,c})^2$ ;
6       for  $j = 1$  to  $T$  do
7         Normalize:  $\hat{X}_{i,j,c} = \frac{X_{i,j,c} - \mu_{i,c}}{\sqrt{\sigma_{i,c}^2 + \epsilon}}$ ;
8         Scale and shift:  $B_{i,j,c} = \gamma_c \hat{X}_{i,j,c} + \beta_c$ ;
9   return  $\mathcal{B}^{(d)}$ ;

```

Similarly, the inverse normalization process restores the original data distribution to the model's output is outlined in Algorithm 3.

**Algorithm 3:** RevIN Denormalization Algorithm.

**Input:** Tensor  $\hat{\mathbf{B}}^{(d)}$  containing  $M$  matrices  
 $\hat{\mathbf{B}}^{(i)} = [\hat{B}_{i,j,c}] \in \mathbb{R}^{N \times F}$ , where  $i = 1, 2, \dots, M$ ;  
 $\gamma_c, \beta_c$  are the learnable parameters for feature  $c$ ;  
 $\epsilon$  is a small constant, typically set to  $1e - 5$ .

**Output:** Tensor  $\hat{\mathbf{Y}}^{(d)}$  containing  $M$  matrices  
 $\hat{\mathbf{Y}}^{(i)} = [\hat{Y}_{i,j,c}] \in \mathbb{R}^{N \times C}$ , where  $i = 1, 2, \dots, M$

```

1 Function RevIN_DeNorm( $\hat{\mathbf{B}}^{(d)}$ ) :
2   for  $i = 1$  to  $M$  do
3     for  $c = 1$  to  $F$  do
4       for  $j = 1$  to  $N$  do
5         Denormalize:
6            $\hat{Y}_{i,j,c} = \sqrt{\sigma_{i,c}^2 + \epsilon} \left( \frac{\hat{B}_{i,j,c} - \beta_k}{\gamma_k} \right) + \mu_{i,c}$ ;
7   return  $\hat{\mathbf{Y}}^{(d)}$ ;

```

**3.2.4. RevIN-TSMixer model**

RevIN serves as both a preprocessing and postprocessing technique, enhancing the robustness of the TSMixer model by normalizing input time series data and reversing the normalization of the model's output. This integration effectively addresses common challenges in real-world time series forecasting, such as distributional shifts and noise, thereby enabling the model to achieve improved forecasting performance. The integration of the RevIN method into the TSMixer model is outlined in Algorithm 4.

**Algorithm 4:** RevIN-TSMixer Model Algorithm.

**Input:** Input tensor  $\mathcal{X}^{(d)}$ ;  $K$ , the number of Mixing Layers used in the TSMixer model.

**Output:** Predicted output tensor  $\hat{\mathbf{Y}}^{(d)}$ .

```

1 Function RevIN_TSMixer( $\mathcal{X}^{(d)}$ ,  $K$ ) :
2    $\mathbf{B}^{(d)} = \text{RevIN\_Norm}(\mathcal{X}^{(d)})$ ; // RevIN Normalization
3    $\hat{\mathbf{B}}^{(d)} = \text{TSMixer}(\mathbf{B}^{(d)}, K)$ ; // Apply TSMixer model using
4    $\hat{\mathbf{Y}}^{(d)} = \text{RevIN\_DenNorm}(\hat{\mathbf{B}}^{(d)})$ ; // RevIN Denormalization
5   return  $\hat{\mathbf{Y}}^{(d)}$ ;

```

This integration of RevIN into the TSMixer model facilitates improved generalization across diverse time series distributions while maintaining computational efficiency, making it particularly suitable for handling complex forecasting scenarios.

**3.3. Probabilistic inventory model**

After determining the customer's demand, companies must maintain sufficient stock to ensure the timely fulfillment of orders. An effective inventory model is essential for establishing management strategies that minimize inventory-related costs while meeting customer demand. This study applies a probabilistic inventory model to address demand uncertainty, where demand fluctuates based on forecasted results generated by RevIN-TSMixer.

**3.3.1. The continuous review policy ( $r, q$ )**

The most common inventory replenishment policies are the periodic review system and the continuous review system. Due to the characteristics of the problem, the continuous review ( $r, q$ ) policy is employed. The ( $r, q$ ) policy, also known as the continuous review reorder point policy, is widely used inventory management strategy that ensures optimal inventory levels by monitoring stock continuously and triggering replenishment based on two parameters (Winston, 2022): reorder point ( $r$ ) is the inventory level at which a new order is placed, order quantity ( $q$ ) is the fixed amount of stock replenished each time an order is placed.

The ( $r, q$ ) policy ensures that inventory is restocked before it is depleted, minimizing the risk of stockouts while avoiding excessive holding costs. The optimal order quantity  $Q^*$  in ( $r, q$ ) policy will be defined by  $\sqrt{\frac{2 \cdot \mu_D \cdot o}{h}}$ , where  $\mu_D$  is the averaged forecasted demand,  $o$  is the fixed costs incurred when an order is placed and  $h$  is the holding cost per unit over a month. The forecasted demand is defined by  $\mu_D = \frac{1}{N} \cdot \sum_{i=1}^N F_i$ , where  $F_i$  is the forecasted demand for the  $i$ -th month, and  $N$  is the total number of months under consideration. The forecasted demand is obtained using our proposed forecasting model in Section 3.2.

**3.3.2. Determination of total costs**

Minimizing the inventory cost is the primary objective of any inventory model. According to the EOQ model, inventory cost consists of ordering costs and holding costs. However, in reality, businesses also face shortage costs when demand exceeds available stock, leading to lost sales or backorders. By incorporating shortage costs into the model, businesses can optimize inventory levels to balance ordering, holding, and shortage costs, ultimately improving supply chain efficiency and customer satisfaction.

The shortage cost per unit  $c_s$  of being overstocked or understocked is typically influenced by customer expectations and market conditions. Customers determine this cost based on their sensitivity to stockouts, willingness to wait for backorders, or their likelihood of switching to competitors. However, it is usually very difficult to determine accurately the cost of being one unit short. Therefore, managers often decide to control shortages by meeting a specified service level  $\alpha$  through the stockout probability  $P(X \geq r)$  in Eq. (14) (Winston, 2022).

$$\alpha = 1 - P(X \geq r) = 1 - \frac{c_h \cdot Q^*}{c_s \cdot \mu_D}, \quad (14)$$

where  $c_h$  is holding cost per unit and  $c_s$  is shortage cost per unit.  $Q^*$  is the optimal order quantity, determined by Eq. (14) with the average forecasted demand  $\mu_D$ .  $P(X \geq r)$  represents the probability that a stockout will occur during the lead time, meaning the demand  $X$  exceeds the order point  $r$ .

Once  $P(X \geq r)$  is known, the corresponding z-score is determined by the standard normal distribution,  $z_\alpha = \Phi^{-1}(\alpha)$ . Based on the estimated service level  $z_\alpha$ , the safety stock  $SS$  and reorder point  $r$  are determined to ensure sufficient inventory during demand fluctuations and lead time.

$$SS = z_\alpha \cdot \sigma_D \cdot \sqrt{L}, \quad (15)$$

$$r = \mu_D \cdot L + SS. \quad (16)$$

Eq. (15) estimates the safety stock  $SS$ , which is an additional amount of inventory kept as a buffer to account for uncertainties in demand over the lead time  $L$  – the time between placing and receiving an order. Eq. (16) calculates the reorder point  $r$ , composed of the average expected demand  $\mu_D$  during the lead time  $L$  and the safety stock  $SS$  to mitigate the risk of shortages.

By leveraging the loss function  $\mathcal{L}(z)$ , the expected number of shortages  $E(S)$  and the corresponding expected shortage costs  $E(c_s)$  can be systematically determined (Zuo-Jun Max Shen, 2019).

$$\mathcal{L}(z) = \int_z^\infty (y - z) \cdot \phi(y) dy, \quad (17)$$

$$E(S) = \mathcal{L}(z) \cdot \sigma_D \cdot \sqrt{L}, \quad (18)$$

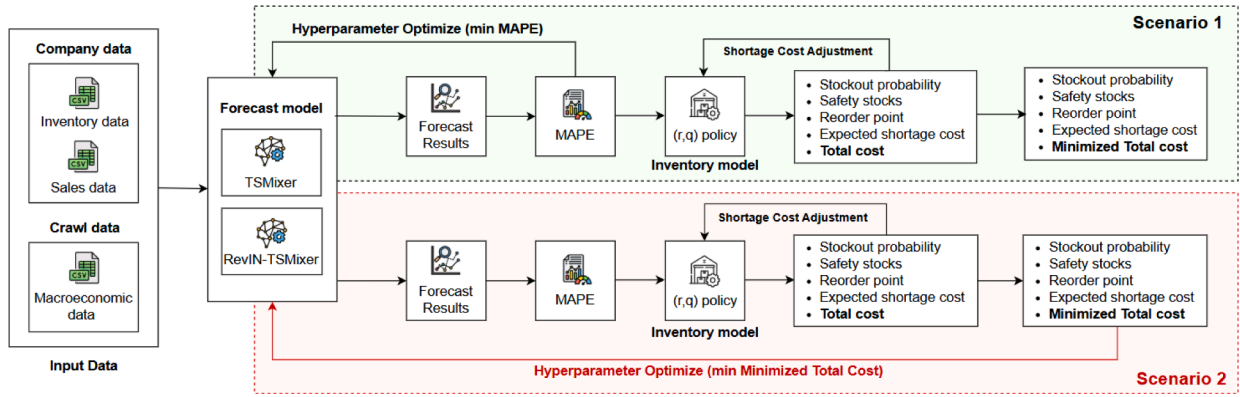


Fig. 3. The process of integrating forecast model and inventory model within these two scenarios.

$$E(c_s) = \frac{c_s \cdot E(S) \cdot \mu_D}{Q^*}. \quad (19)$$

In Eq. (17), loss function  $\mathcal{L}(z)$  represents the probability-weighted expected shortages, incorporating the cumulative distribution function  $\phi(z)$ , where  $\phi(z) = \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{z^2}{2}}$ . Eq. (18) expresses the expected number of shortages  $E(S)$ , where the term  $\sigma_D \cdot \sqrt{L}$  accounts for the variability in demand during the lead time, making  $E(S)$  a measure of expected shortfalls in stock due to demand fluctuations. Eq. (19) quantifies the total cost incurred due to shortages during the lead time.

Then, the total cost  $TC$ , which includes inventory costs  $c_i$ —comprising of ordering and holding costs—and expected shortage costs  $E(c_s)$  is defined as follow:

$$c_o = \frac{\mu_D}{Q^*} \cdot o, \quad (20)$$

$$c_h = \left( \frac{Q^*}{2} + \max(SS, 0) \right) \cdot h, \quad (21)$$

$$TC = c_o + c_h + E(c_s). \quad (22)$$

Eqs. (20) and (21) defined the ordering costs and holding costs, respectively. In Eq. (21), the function  $\max(SS, 0)$  ensures that only positive safety stock values contribute to holding costs, preventing negative values from affecting cost calculations. Then, Eq. (22) defines the total cost  $TC$  as the sum of ordering cost, holding cost, and the expected shortage cost defined in Eq. (19).

By integrating ordering, holding, and shortage costs by determining the desired service level—which represents the probability of avoiding stockout—we apply a comprehensive framework for optimizing inventory management. This framework ensures a strategic balance between cost efficiency and service level requirements, minimizing total costs while maintaining sufficient stock to meet customer demand.

In this study, we propose integrating the specific forecasting model with the inventory model. To evaluate the effectiveness of this integration, we conducted experiments based on two scenarios introduced in Section 3.1. The process of integrating forecasting and inventory management within these two scenarios is illustrated in Fig. 3.

## 4. Experiments

### 4.1. Dataset

The dataset used in this study was collected from a manufacturing company operating in the logistics sector and headquartered in Vietnam. It covers the period from 2015 to 2024. The company's internal data includes information on the consumption demand of various product lines, which is aggregated to form a monthly time series representing the total product demand of the firm. In addition, the internal dataset incorporates important variables such as

monthly promotional spending and estimated demand of competitors, which were collected from market reports and commercial intelligence sources.

To enrich the dataset and account for macro-level influences, we also incorporated several macroeconomic indicators. These indicators were obtained from Vietstock<sup>1</sup>, a reputable financial and economic data provider in Vietnam.

In total, the dataset consists of 120 data points, corresponding to 120 months (from January 2015 to December 2024), and includes 12 carefully selected features from both internal business operations and macroeconomic environments. A detailed description of these features is presented in Table 4. The datasets used in this study is available at our GitHub repository<sup>2</sup>.

### 4.2. Baselines

To comprehensively assess the effectiveness of the proposed model on real-world datasets, we selected several widely-used and high-performing baseline models in time series forecasting, including Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), Deep Neural Networks (DNN), and eXtreme Gradient Boosting (XGBoost - XGB). These models were chosen to ensure objective and reliable comparisons, thereby highlighting the proposed model's improvements in both performance and generalization. Beyond serving as performance benchmarks, the baselines also help validate the practical value of the proposed approach in inventory management. Through direct comparison, this study aims to demonstrate that the proposed model delivers superior forecasting accuracy and contributes meaningfully to data-driven inventory decision-making.

### 4.3. Exploratory data analysis (EDA)

To gain initial insights into the dataset, we performed an exploratory data analysis (EDA) on both the company-specific and macroeconomic attributes. Fig. 4 illustrates the time series visualization of key features, where the company-level indicators are shown in blue and macroeconomic variables in orange. The visualization highlights different seasonal patterns and trends across feature groups, suggesting a potentially complex interaction between firm-specific behavior and broader economic conditions.

To support demand modeling in the context of inventory control, we investigated the empirical distribution of the demand-related variables. In practice, inventory decisions often assume specific probability distributions such as normal, uniform, triangular, or discrete forms. Among

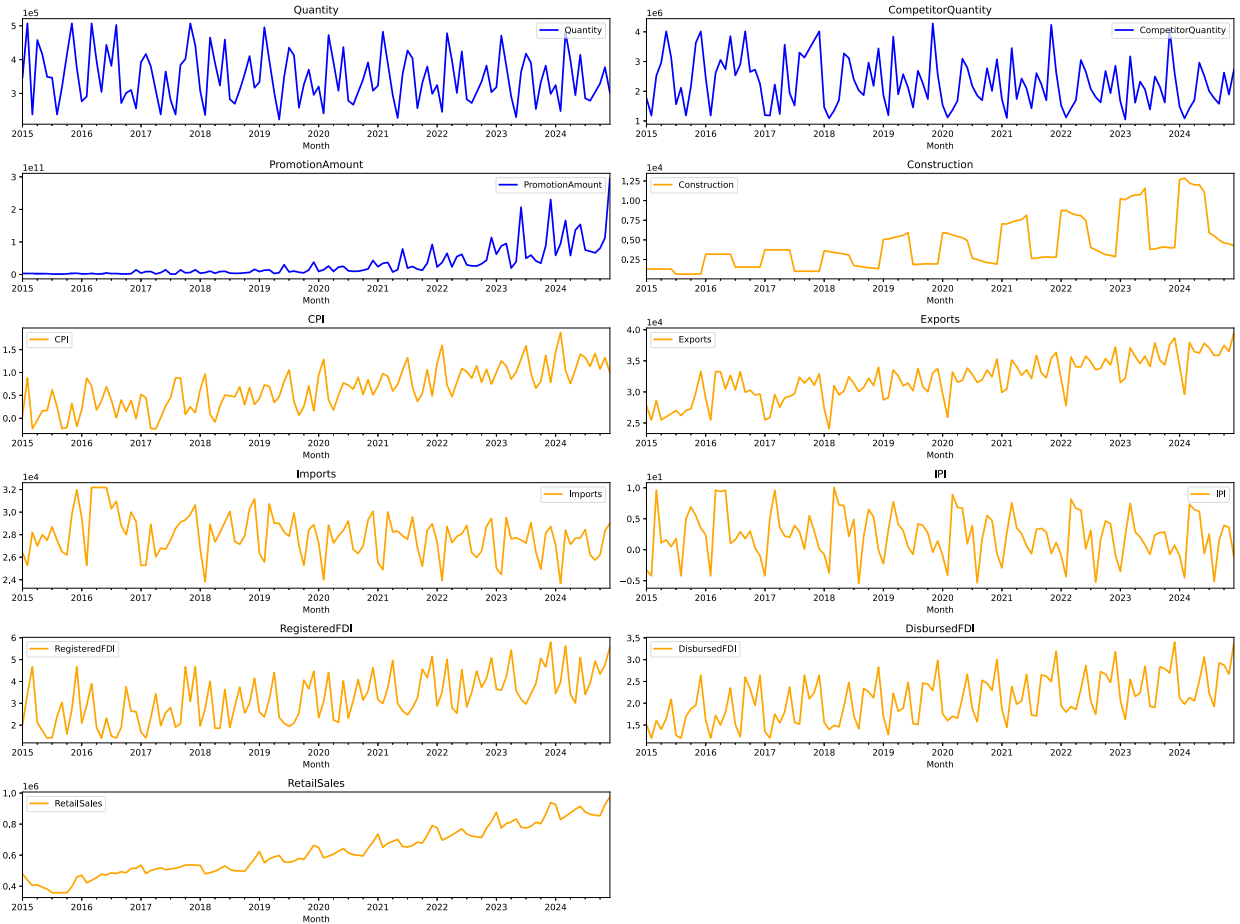
<sup>1</sup> Vietstock website: <https://finance.vietstock.vn/du-lieu-vi-mo>

<sup>2</sup> Our GitHub repository: [https://github.com/anhnguyenthingoc/tsi\\_tsmixer\\_forecasting\\_inventory/tree/main](https://github.com/anhnguyenthingoc/tsi_tsmixer_forecasting_inventory/tree/main)

**Table 4**

Attribute names, descriptions, and variable types in the dataset.

| No. | Attribute name     | Description                                                                                                      | Feature | Response |
|-----|--------------------|------------------------------------------------------------------------------------------------------------------|---------|----------|
| 1   | Month              | Month within the period from January 2015 to December 2024.                                                      | ✓       | ✗        |
| 2   | Quantity           | Monthly total sales quantity aggregated across all products of the company.                                      | ✗       | ✓        |
| 3   | CompetitorQuantity | Estimated monthly sales quantity of competing firms in the same product categories.                              | ✓       | ✗        |
| 4   | PromotionAmount    | Monthly promotional expenditure related to product sales.                                                        | ✓       | ✗        |
| 5   | Construction       | Number of newly initiated construction projects, including residential and public buildings.                     | ✓       | ✗        |
| 6   | CPI                | Consumer Price Index reflecting the general price level changes of goods and services over time.                 | ✓       | ✗        |
| 7   | Exports            | Total monthly export value, excluding international insurance and transportation fees.                           | ✓       | ✗        |
| 8   | Imports            | Total monthly import value, including goods, insurance, and transportation costs.                                | ✓       | ✗        |
| 9   | IPI                | Industrial Production Index (IPI) comparing current production volume with the same period in the previous year. | ✓       | ✗        |
| 10  | RegisteredFDI      | Total registered value of foreign direct investment (FDI) during the period.                                     | ✓       | ✗        |
| 11  | DisbursedFDI       | Actual utilized value of foreign direct investment in the economy during the period.                             | ✓       | ✗        |
| 12  | RetailSales        | Total retail sales value representing goods and services sold directly to consumers.                             | ✓       | ✗        |

**Fig. 4.** Time series visualization of dataset attributes: company data (Blue) and macroeconomic data (Orange).

them, the normal distribution is frequently used for setting service levels in inventory models, particularly in determining safety stock and reorder points.

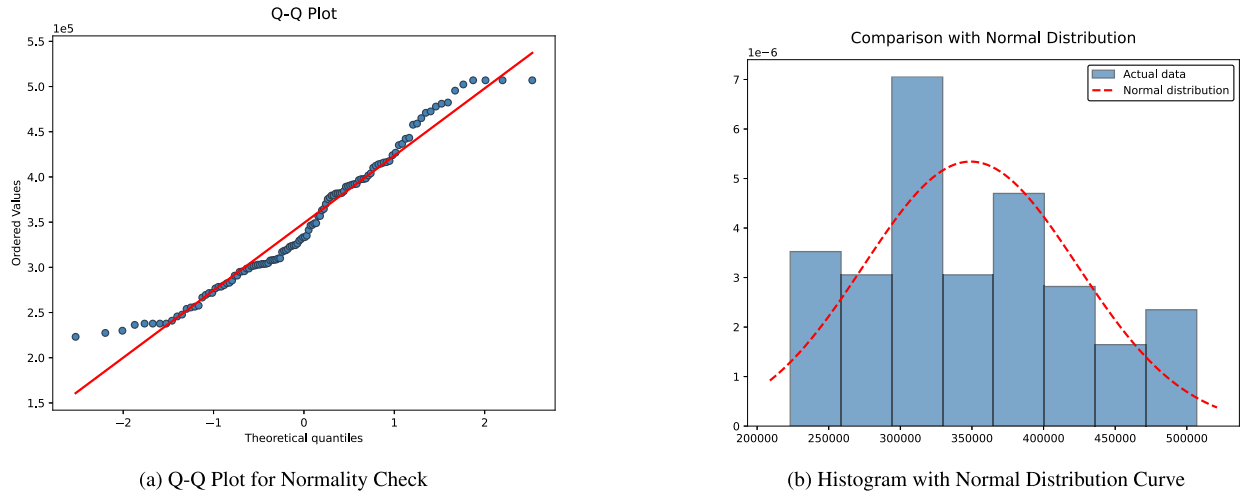
Fig. 5 provides a visual assessment of the distributional shape using a Q-Q plot and a histogram with a fitted normal curve. To confirm this observation, we conducted a Kolmogorov-Smirnov (K-S) test for normality, obtaining a p-value of 0.1742. As this exceeds the 0.05 significance level, we fail to reject the null hypothesis, suggesting that the demand variable reasonably follows a normal distribution. This result supports the validity of applying a normality-based approach to service level selection in the methodology section.

In addition to historical demand, we incorporate a set of macroeconomic indicators into the forecasting model. These indicators serve as contextual features that enhance the forecasting model's responsive-

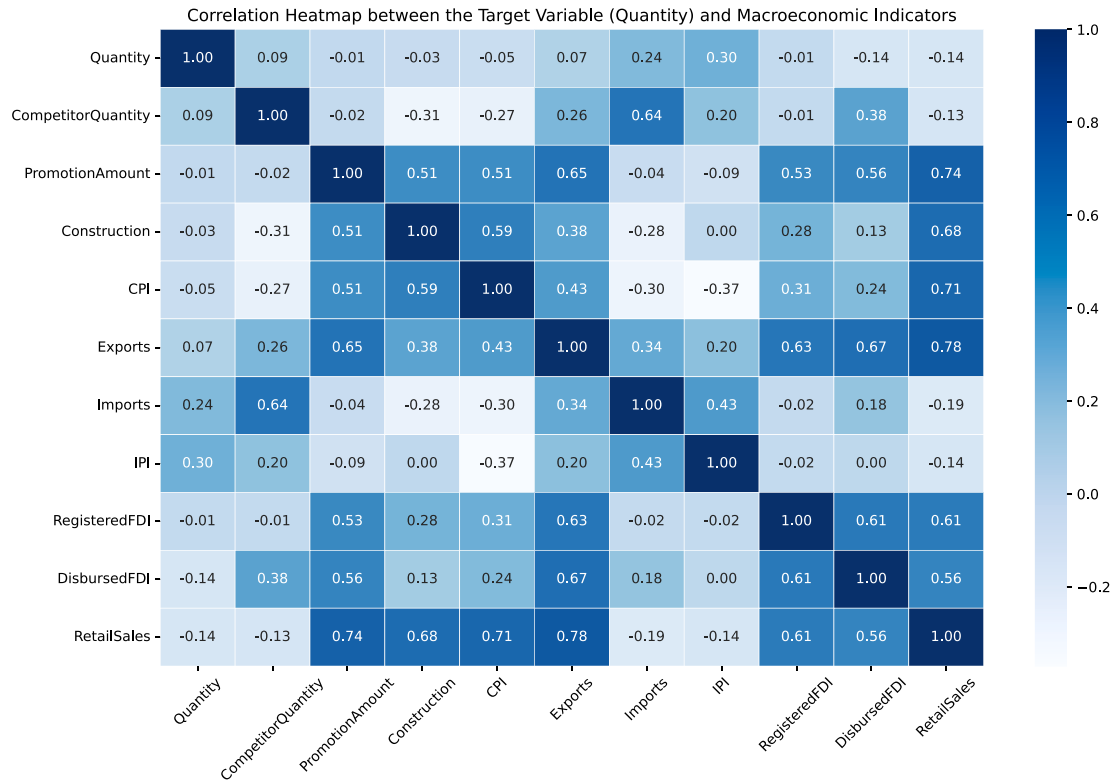
ness to external market conditions. The inclusion of features such as the Industrial Production Index (IPI), Imports, and Consumer Price Index (CPI) is motivated by their demonstrated correlation with demand (Fig. 6). They capture broader economic dynamics that influence consumer behavior, production activity and supply chain conditions. These factors are often overlooked when relying solely on historical demand.

#### 4.4. Experimental setup

In the revised experiments, the dataset was split into training, validation, and test sets with a ratio of 80:10:10. The first 80 % of the chronological data was used for training the models, the next 10 % for validation to tune hyperparameters and prevent overfitting, and the final 10 % was reserved as the test set for unbiased evaluation of model per-



**Fig. 5.** Visual assessment of the normality assumption using Q-Q plot and histogram. (a) Q-Q plot for normality check, (b) histogram with normal distribution curve.



**Fig. 6.** Correlation heatmap between the target variable (quantity) and macroeconomic indicators.

formance. This splitting strategy ensures that temporal dependencies are preserved and that the model's generalization performance is evaluated in a reliable and unbiased manner.

#### 4.5. Feature selection

The feature selection process aims to identify or reduce the number of attributes in a feature dataset to enhance model performance. Feature selection is commonly employed in machine learning and pattern recognition applications to improve data interpretability, increase prediction accuracy, and reduce computational complexity.

In this study, the Random Forest (RF) method (Barros et al., 2021) and the XGBoost (XGB) method (Zhang & Jánošík, 2024) were used to evaluate the importance of features related to the target variable,

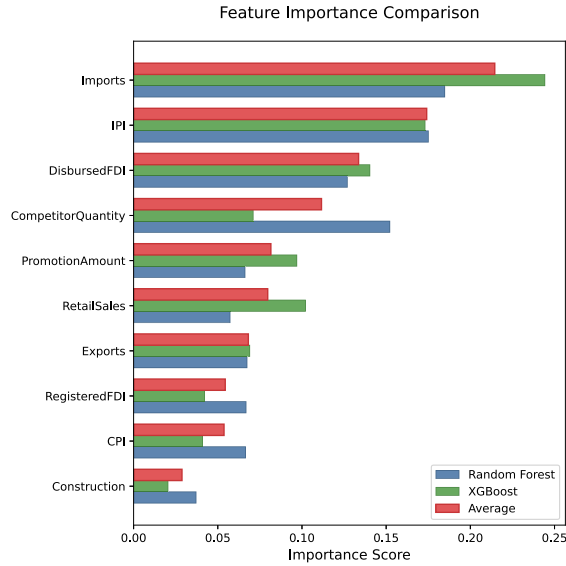
“Quantity”. The importance scores of each feature were calculated separately for both models and then averaged to enhance robustness and reduce potential bias from individual algorithms. Table 5 and Fig. 7 show a comparison of feature importance scores based on real-world data. According to these results, the selected feature columns are “Imports”, “IPI”, “DisbursedFDI”, “CompetitorQuantity”, “PromotionAmount”, with “Quantity” used as the target variable.

#### 4.6. Hyperparameter optimization

To ensure optimal performance and enable reproducibility, we conducted an extensive hyperparameter search for the TSMixer model. The key hyperparameters considered include the number of mixing blocks (n\_block), the feed-forward dimension (ff\_dim), the activation

**Table 5**  
Feature importance comparison (numerical results).

| Feature            | RF     | XGB    | Avg    |
|--------------------|--------|--------|--------|
| Imports            | 0.1848 | 0.2442 | 0.2145 |
| IPI                | 0.1751 | 0.1731 | 0.1741 |
| DisbursedFDI       | 0.1269 | 0.1403 | 0.1336 |
| CompetitorQuantity | 0.1522 | 0.0710 | 0.1116 |
| PromotionAmount    | 0.0661 | 0.0969 | 0.0815 |
| RetailSales        | 0.0573 | 0.1021 | 0.0797 |
| Exports            | 0.0673 | 0.0690 | 0.0682 |
| RegisteredFDI      | 0.0668 | 0.0421 | 0.0544 |
| CPI                | 0.0665 | 0.0409 | 0.0537 |
| Construction       | 0.0371 | 0.0204 | 0.0287 |



**Fig. 7.** Feature importance comparison (Visualization).

function, the input sequence length (seq\_len), dropout rate (dropout), batch size (batch\_size), and learning rate (learning\_rate). The number of hidden layers in the model corresponds to the number of TSMixer blocks (n\_block), where each block comprises temporal and feature mixing components, followed by a fully-connected feed-forward layer. The ff\_dim controls the number of neurons in these layers, effectively determining the model's capacity. We explored n\_block values of {1, 2, 3} and ff\_dim values of {8, 16, 32, 64, 128}.

For the activation function, we employed ReLU (Rectified Linear Unit) across all configurations due to its simplicity, computational efficiency, and ability to mitigate the vanishing gradient problem. This choice has been explicitly included in the updated tables to enhance clarity.

These values are explored using a Grid Search algorithm (Abdelkader et al., 2025; Belete & Huchaiah, 2022) to identify the optimal configuration that minimizes performance metrics such as MAPE or Minimized Total Cost. The final hyperparameter choices used for each variant of TSMixer are summarized in Tables 6–8, which have been updated to include the activation function column for completeness.

Table 9 compares the training and testing performance of the RevIN-TSMixer and TSMixer models across two scenarios using Mean Squared Error (MSE) and Mean Absolute Error (MAE) as evaluation metrics. Overall, RevIN-TSMixer consistently outperforms TSMixer on both datasets. Specifically, in Scenario 1, RevIN-TSMixer achieves training MSE and MAE of 0.0597 and 0.1890, respectively, and testing MSE and MAE of 0.0785 and 0.2127. In Scenario 2, the model maintains strong performance with training MSE and MAE of 0.0575 and 0.1854, and testing MSE and MAE slightly increasing to 0.0863 and 0.2186, indicating stable generalization. In contrast, TSMixer exhibits

significantly poorer performance. For Scenario 1, its training MSE and MAE are 1.1237 and 0.8279, but these metrics rise sharply on the test set to 3.4210 and 1.1931. A similar trend is observed in Scenario 2, with training MSE and MAE at 1.0155 and 0.8158, and a substantial increase on the test set to 5.1648 and 1.3900. In summary, RevIN-TSMixer demonstrates superior accuracy and robustness across both scenarios, while TSMixer struggles to maintain performance, especially in Scenario 2.

#### 4.7. Evaluation metrics

In time series forecasting, evaluating the performance of a model is crucial to ensure that the forecasts it produces are accurate and reliable. Common metrics used to evaluate time series forecasting models include MSE, MAE, RMSE, and MAPE. These metrics are defined in Eq. (23):

$$\begin{aligned} \text{MSE} &= \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2, & \text{MAE} &= \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|, \\ \text{RMSE} &= \sqrt{\text{MSE}}, & \text{MAPE} &= \frac{1}{n} \sum_{i=1}^n \frac{|Y_i - \hat{Y}_i|}{|Y_i|} \times 100, \end{aligned} \quad (23)$$

where  $n$  is the number of samples in the dataset,  $Y_i$  is the actual value of the  $i$ -th sample and  $\hat{Y}_i$  is the predicted value corresponding to the  $i$ -th sample.

In addition to traditional metrics such as MSE, MAE, RMSE, and MAPE, Scenario 2 incorporates the Total Cost metric as a key evaluation criterion. The formula for calculating Total Cost is presented in Eq. (22). Given that shortage levels fluctuate, the evaluation focuses on optimizing hyperparameters by minimizing the Total Cost to adapt to varying demand conditions. By integrating this metric, the evaluation framework not only assesses forecast accuracy but also accounts for the financial impact of forecasting errors such as overstocking and stock-outs. This holistic approach enhances resource allocation, optimizes inventory decisions, and improves overall decision-making in real-world inventory management applications.

#### 4.8. Results and discussion

In the case studied, the company fixed the lead time  $L$  to two months (that is,  $L = 2$ ) and maintained constant ordering and holding costs according to company regulations. The shortage cost per unit  $c_s$  is considered a variable, constrained within a specified range, that is,  $c_s \in (0; 10]$ . This range allows the study to analyze how different levels of shortage cost impact inventory decisions, then identify trends in stockout probability, safety stock, reorder point, expected shortage cost and total cost.

##### 4.8.1. Evaluation of forecasting models: accuracy and cost implications

A comparative analysis of four models, including our proposed model RevIN-TSMixer, TSMixer, LSTM and GRU, was evaluated under two distinct scenarios: error optimization—minimizing MAPE—and cost optimization—minimizing the annual total cost, as summarized in Tables 10 and 11. The evaluation criteria include MSE, MAE, RMSE, MAPE, minimized total cost  $TC_{min}$  and corresponding optimal shortage cost per unit  $c_s^*$ . Here,  $TC_{min}$  is determined by the total cost function, minimized over a range of possible  $c_s$ . It can be expressed as  $TC_{min} = \min_{c_s} TC(c_s)$ , where  $TC(c_s)$  represents the total cost as a function of  $c_s$ . Since different models yield different demand forecasting results, their corresponding total costs will also vary due to variability in  $c_s$ . The optimal  $c_s^*$  represents the cost parameter at which  $TC$  is minimized, ensuring a fair and meaningful comparison across different forecasting models.

There is a significant difference between forecasting performance in the two scenarios. In Scenario 1, which focuses on error optimization, our proposed model, RevIN-TSMixer, achieves the lowest MSE, MAE, and RMSE, indicating superior forecasting accuracy (Table 10). Notably, RevIN-TSMixer outperforms all baseline models in both forecasting accuracy and total cost minimization. Compared to the standard TSMixer,

**Table 6**  
Hyperparameter optimize results for RevIN-TSMixer models (The best numbers are shown in **bold**).

| pred_len<br><i>H</i> | seq_len<br>1,2,...,12 | n_block<br>1,2,3 | dropout<br>0.1, 0.3, ..., 0.9 | batch_size<br>1, 2, 3, 4 | ff_dim<br>8, 16, 32, 64,128 | learning_rate<br>1e-4, 1e-5 | activation | MAPE (%)        | Minimized Total Cost |
|----------------------|-----------------------|------------------|-------------------------------|--------------------------|-----------------------------|-----------------------------|------------|-----------------|----------------------|
| 3                    | 1                     | 1                | 0.1                           | 1                        | 8                           | 1e-4                        | ReLU       | 33.12746        | 482,069.61           |
| 3                    | 1                     | 1                | 0.1                           | 1                        | 8                           | 1e-5                        | ReLU       | 33.12886        | 482,085.95           |
| ⋮                    | ⋮                     | ⋮                | ⋮                             | ⋮                        | ⋮                           | ⋮                           | ⋮          | ⋮               | ⋮                    |
| 3                    | 6                     | 1                | 0.1                           | 4                        | 128                         | 1e-4                        | ReLU       | <b>14.26424</b> | 306,701.95           |
| ⋮                    | ⋮                     | ⋮                | ⋮                             | ⋮                        | ⋮                           | ⋮                           | ⋮          | ⋮               | ⋮                    |
| 3                    | 9                     | 2                | 0.1                           | 2                        | 128                         | 1e-4                        | ReLU       | 18.64721        | <b>299,738.43</b>    |
| ⋮                    | ⋮                     | ⋮                | ⋮                             | ⋮                        | ⋮                           | ⋮                           | ⋮          | ⋮               | ⋮                    |
| 3                    | 12                    | 3                | 0.9                           | 4                        | 128                         | 1e-4                        | ReLU       | 25.37053        | 407,019.18           |
| 3                    | 12                    | 3                | 0.9                           | 4                        | 128                         | 1e-5                        | ReLU       | 42.27403        | 523,332.92           |

**Table 7**  
Hyperparameter optimize results for TSMixer models (The best numbers are shown in **bold**).

| pred_len<br><i>H</i> | seq_len<br>1,2,...,12 | n_block<br>1,2,3 | dropout<br>0.1, 0.3, ..., 0.9 | batch_size<br>1, 2, 3, 4 | ff_dim<br>8, 16, 32, 64,128 | learning_rate<br>1e-4, 1e-5 | activation | MAPE (%)         | Minimized Total Cost |
|----------------------|-----------------------|------------------|-------------------------------|--------------------------|-----------------------------|-----------------------------|------------|------------------|----------------------|
| 3                    | 1                     | 1                | 0.1                           | 1                        | 8                           | 1e-4                        | ReLU       | 16.22197         | 343,213.47           |
| 3                    | 1                     | 1                | 0.1                           | 1                        | 8                           | 1e-5                        | ReLU       | 16.24722         | 344,247.31           |
| ⋮                    | ⋮                     | ⋮                | ⋮                             | ⋮                        | ⋮                           | ⋮                           | ⋮          | ⋮                | ⋮                    |
| 3                    | 6                     | 3                | 0.3                           | 4                        | 128                         | 1e-4                        | ReLU       | 18.261066        | <b>309,500.35</b>    |
| ⋮                    | ⋮                     | ⋮                | ⋮                             | ⋮                        | ⋮                           | ⋮                           | ⋮          | ⋮                | ⋮                    |
| 3                    | 9                     | 3                | 0.3                           | 4                        | 64                          | 1e-4                        | ReLU       | <b>14.964953</b> | 366,377.40           |
| ⋮                    | ⋮                     | ⋮                | ⋮                             | ⋮                        | ⋮                           | ⋮                           | ⋮          | ⋮                | ⋮                    |
| 3                    | 12                    | 3                | 0.9                           | 4                        | 128                         | 1e-4                        | ReLU       | 85.70003         | 753,648.73           |
| 3                    | 12                    | 3                | 0.9                           | 4                        | 128                         | 1e-5                        | ReLU       | 96.02774         | 793,984.03           |

**Table 8**  
Hyperparameter search space and the best hyperparameter configuration for each models.

| Scenario 1. Optimize MAPE       |                      |                       |                  |                               |                          |                              |                             |            |  |
|---------------------------------|----------------------|-----------------------|------------------|-------------------------------|--------------------------|------------------------------|-----------------------------|------------|--|
| Model                           | pred_len<br><i>H</i> | seq_len<br>1,2,...,12 | n_block<br>1,2,3 | dropout<br>0.1, 0.3, ..., 0.9 | batch_size<br>1, 2, 3, 4 | ff_dim<br>8, 16, 32, 64, 128 | learning_rate<br>1e-4, 1e-5 | activation |  |
| RevIN-TSMixer                   | 3                    | 6                     | 1                | 0.1                           | 4                        | 128                          | 1e-4                        | ReLU       |  |
| TSMixer                         | 3                    | 9                     | 3                | 0.3                           | 4                        | 64                           | 1e-4                        | ReLU       |  |
| Scenario 2. Optimize Total Cost |                      |                       |                  |                               |                          |                              |                             |            |  |
| Model                           | pred_len<br><i>H</i> | seq_len<br>1,2,...,12 | n_block<br>1,2,3 | dropout<br>0.1, 0.3, ..., 0.9 | batch_size<br>1, 2, 3, 4 | ff_dim<br>8, 16, 32, 64, 128 | learning_rate<br>1e-4, 1e-5 | activation |  |
| RevIN-TSMixer                   | 3                    | 9                     | 2                | 0.1                           | 2                        | 128                          | 1e-4                        | ReLU       |  |
| TSMixer                         | 3                    | 6                     | 3                | 0.3                           | 4                        | 128                          | 1e-4                        | ReLU       |  |

**Table 9**  
Train and test performance comparison of different models.

| Model                      | Train performance |        | Test performance |        |
|----------------------------|-------------------|--------|------------------|--------|
|                            | MSE               | MAE    | MSE              | MAE    |
| RevIN-TSMixer (Scenario 1) | 0.0597            | 0.1890 | 0.0785           | 0.2127 |
| RevIN-TSMixer (Scenario 2) | 0.0575            | 0.1854 | 0.0863           | 0.2186 |
| TSMixer (Scenario 1)       | 1.1237            | 0.8279 | 3.4210           | 1.1931 |
| TSMixer (Scenario 2)       | 1.0155            | 0.8158 | 5.1648           | 1.3900 |

integrating RevIN into TSMixer enhances forecasting accuracy by addressing distributional shifts in time series data. RevIN approach helps stabilize forecast results by normalizing input data at both training and inference stages, reducing the impact of non-stationary and seasonality. This capability contributes directly to RevIN-TSMixer's superior forecasting performance compared to the original TSMixer model and other DL-based models.

In Scenario 2, which focuses on cost optimization, all models achieve lower total costs at the optimal shortage cost compared to their performance in Scenario 1 despite exhibiting lower forecasting accuracy

(Table 11). This highlights the effectiveness of incorporating cost considerations directly into the forecasting process. Therefore, it enables models to make more balanced and cost-aware decisions that better align with inventory management objectives, ultimately leading to improved overall cost efficiency.

In both scenarios, our proposed model, RevIN-TSMixer, and other baseline models achieved a lower total inventory cost  $TC_{min}$  compared to the actual demand-based benchmark. The actual demand-based benchmark is estimated based on our inventory model with actual input demand from the company. This improvement highlights the value of integrating forecasting into inventory decision-making. Notably, the RevIN-TSMixer model outperforms the others in terms of inventory performance, demonstrating its superior ability to balance forecasting accuracy with cost optimization.

#### 4.8.2. Inventory performance

To evaluate whether these models maintain their effectiveness in inventory performance, we conduct experiments using the probabilistic inventory model outlined in Section 3.3. The results include the stockout probability  $P(X > r)$ , safety stock  $SS$ , reorder point  $r$ , expected short-

**Table 10**

Experimental results comparison across models and actual value in error optimization scenario (The best value in each row is in **bold**, and the second best is underlined).

| Model                         | MSE           | MAE    | RMSE   | MAPE (%)        | $TC_{min}$        | $c_s^*$ |
|-------------------------------|---------------|--------|--------|-----------------|-------------------|---------|
| RevIN-TSMixer                 | 3,163,003,683 | 48,707 | 56,241 | <b>14.26424</b> | <b>306,701.95</b> | 1.54    |
| TSMixer                       | 5,184,568,547 | 53,037 | 72,004 | 14.96495        | 366,377.40        | 1.56    |
| LSTM                          | 5,737,071,119 | 56,054 | 75,743 | 15.85101        | 361,613.83        | 1.55    |
| GRU                           | 4,337,176,759 | 53,431 | 65,857 | <u>14.69235</u> | 318,292.91        | 1.64    |
| DNN                           | 3,978,752,915 | 53,689 | 63,077 | 15.33763        | 330,017.19        | 1.55    |
| Actual demand-based benchmark | –             | –      | –      | –               | 412,268.74        | 1.54    |

**Table 11**

Experimental results comparison across models and actual value in cost optimization scenario (The best value in each row is in **bold**, and the second best is underlined).

| Model                         | MSE            | MAE     | RMSE    | MAPE (%)        | $TC_{min}$        | $c_s^*$ |
|-------------------------------|----------------|---------|---------|-----------------|-------------------|---------|
| RevIN-TSMixer                 | 5,265,207,070  | 62,405  | 72,562  | 18.64721        | <b>299,738.43</b> | 1.52    |
| TSMixer                       | 4,821,664,902  | 60,287  | 69,438  | <u>18.26107</u> | 309,500.35        | 1.52    |
| LSTM                          | 6,499,960,910  | 64,828  | 80,622  | 17.94816        | 312,186.14        | 1.67    |
| GRU                           | 4,584,916,537  | 53,981  | 67,712  | <b>15.30066</b> | 312,123.68        | 1.61    |
| DNN                           | 19,743,659,877 | 127,436 | 140,512 | 36.98032        | 313,036.30        | 1.96    |
| Actual demand-based benchmark | –              | –       | –       | –               | 412,268.74        | 1.54    |

age cost  $E(c_s)$ , and total cost  $TC$  under varying shortage cost per unit. These metrics are calculated using the actual demand, the forecasted demand from our proposed RevIN-TSMixer and other baseline models, allowing for a comparison of inventory performance across different demand inputs. The experiments are conducted across two scenarios: error optimization and cost optimization.

The common finding across all experiments is that every forecasting model outperforms the inventory performance computed from the actual demand benchmark. This result highlights the effectiveness of the forecasting model in inventory management. It shows that even imperfect forecasts can lead to more cost-effective inventory decisions compared to relying solely on realized demand. By predicting demand, these forecasting models enable proactive inventory control, reducing total cost, minimizing stockouts and optimizing safety stock levels more effectively than original firms' approaches.

The experiment results across two scenarios are consistent with fundamental inventory theory and validate the operational relevance of the forecasting models:

- As shortage cost increases, all models exhibit a notable decrease in stockout probability (Fig. 8), demonstrating that the inventory system becomes more conservative to avoid high penalty costs.
- As shortage costs increase, both safety stock and reorder point tend to rise (Figs. 9 and 10). It reflects the model's behavior to hold more stock as the penalty for stockouts becomes more severe.
- Higher shortage costs force firms to hold more safety stock, reducing the likelihood of stockouts and, consequently, lowering expected shortage costs (Fig. 11)
- The U-shaped pattern of total cost suggests that at very low shortage costs, firms tend to hold excessive inventory to minimize the risk of stockout, leading to high total costs. As shortage costs increase, firms optimize stock levels, reducing total cost until a threshold is reached, beyond which the cost of stockouts outweighs the benefit of lower inventory (Fig. 12)

However, there is a significant difference between the two scenarios, which highlights the advantage of our proposed forecasting framework in evaluating inventory performance. In Scenario 1, where the forecasting process and inventory evaluation are treated as sequential stages, the differences in inventory performance such as stockout probability, safety stock between all models and actual demand benchmark become negligible as the shortage cost increases. This suggests that when cost considerations are not embedded within the forecasting model, improve-

ments in forecasting accuracy alone have limited impact on inventory performance, especially under high penalty conditions.

In contrast, Scenario 2 demonstrates the efficiency of our proposed forecast framework. All models show clear divergence from the actual demand benchmark across all inventory performance metrics. Notably, in terms of total cost, this gap becomes increasingly evident as the shortage cost rises, highlighting the effectiveness of integrating inventory optimization directly into the forecasting model. This integration enables the models to make cost-aware predictions that lead to more efficient inventory decisions, particularly under conditions with high stockout penalties.

In this context, our proposed model, RevIN-TSMixer, still demonstrates the superior inventory performance as shortage cost per unit increases. Despite the overall shift toward more conservative inventory strategies under higher cost pressures, RevIN-TSMixer consistently outperforms other models by achieving lower total cost and reduced stockout probability. This highlights its robustness and effectiveness in aligning forecasting outputs with cost-efficient inventory decisions, even in high-penalty scenarios.

## 5. Discussions

### 5.1. Contributions of the study

This research makes several significant contributions to both theory and practice in supply chain management and forecasting. First, the integration of the RevIN normalization approach with the TSMixer model presents an advanced solution that effectively handles non-stationary and fluctuating demand patterns, contributing to enhanced forecasting robustness and accuracy. Second, by explicitly incorporating the inventory cost optimization into the forecasting process, the study advances existing models that typically focus solely on accuracy without considering broader financial implications. Lastly, the proposed RevIN-TSMixer model offers a practical framework for managers seeking a balanced approach between cost management and service quality under uncertain demand conditions, enriching the set of tools available for strategic inventory management.

### 5.2. Research limitations

Although this study has demonstrated positive outcomes by proposing the RevIN-TSMixer model effectively integrating demand forecasting and inventory management, several limitations need to be acknowledged. First, the study focuses only on data for a single product from

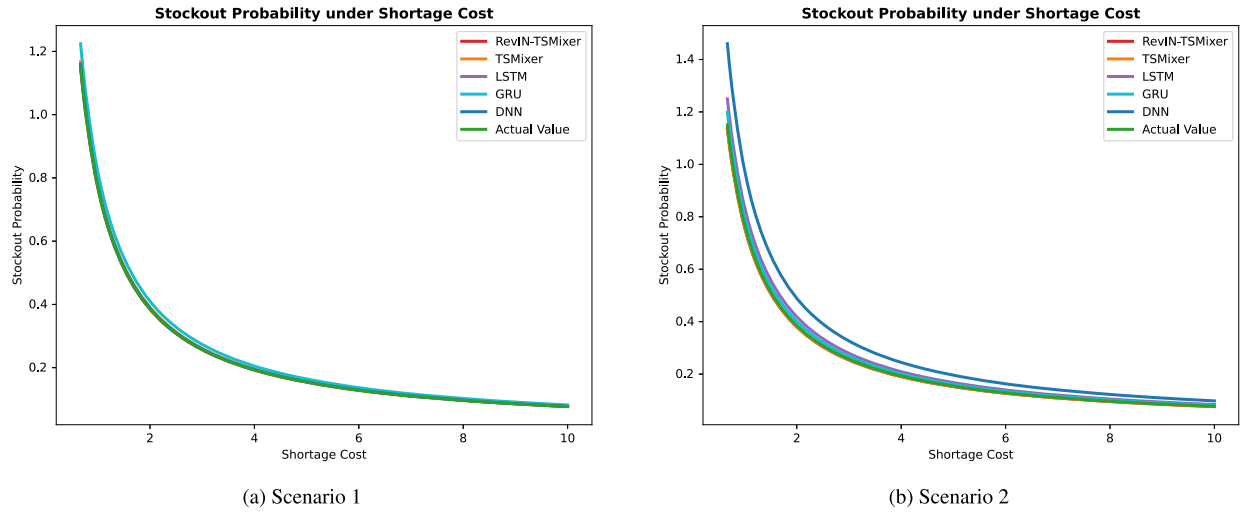


Fig. 8. Stockout probability under varying shortage cost per unit across models.

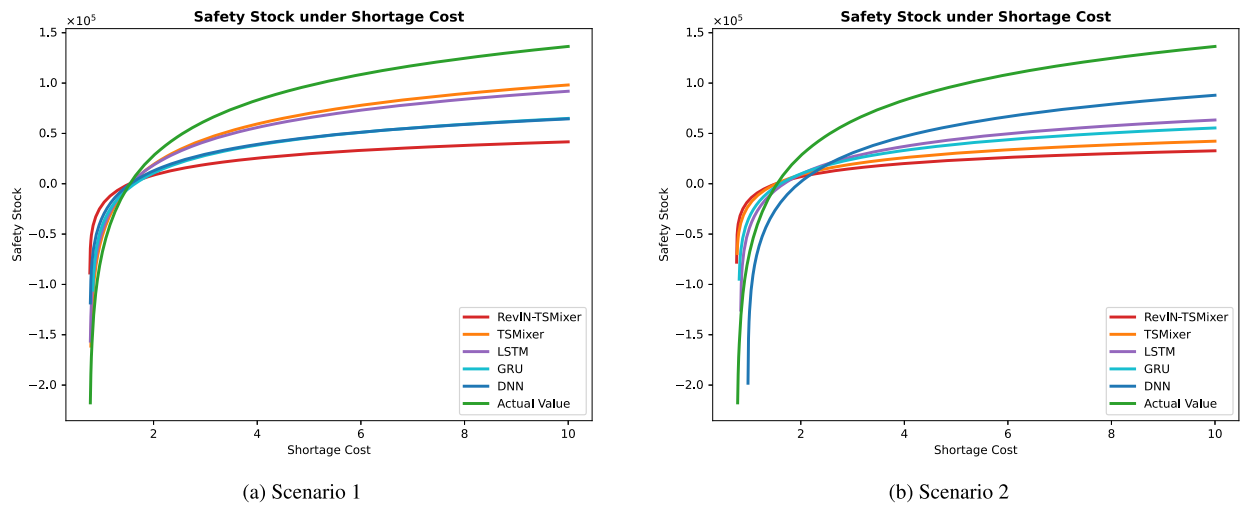


Fig. 9. Safety stocks under varying shortage cost per unit across models.

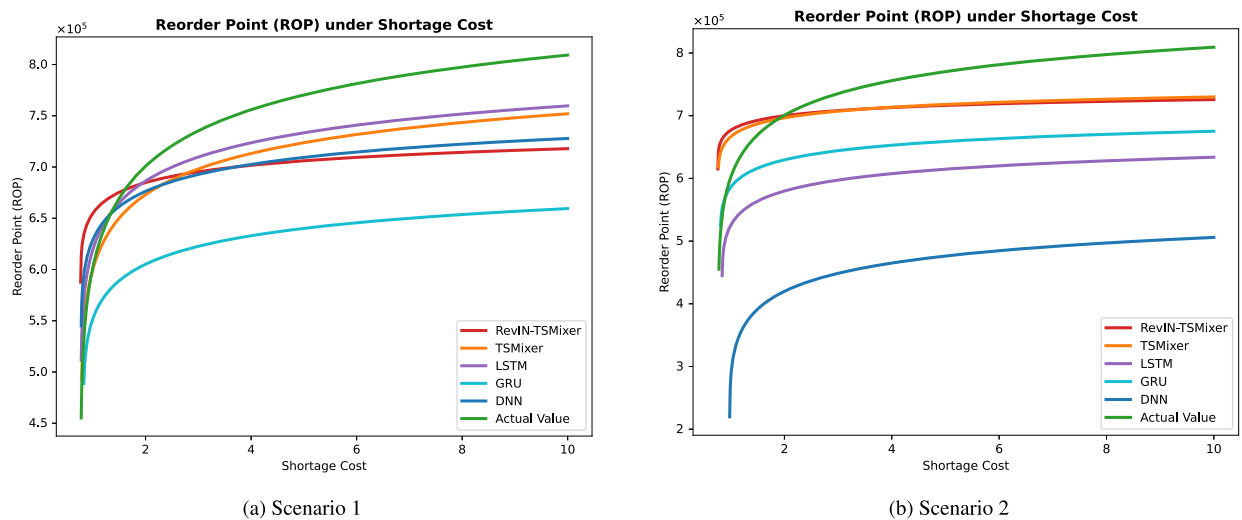


Fig. 10. Reorder point under varying shortage cost per unit across models.

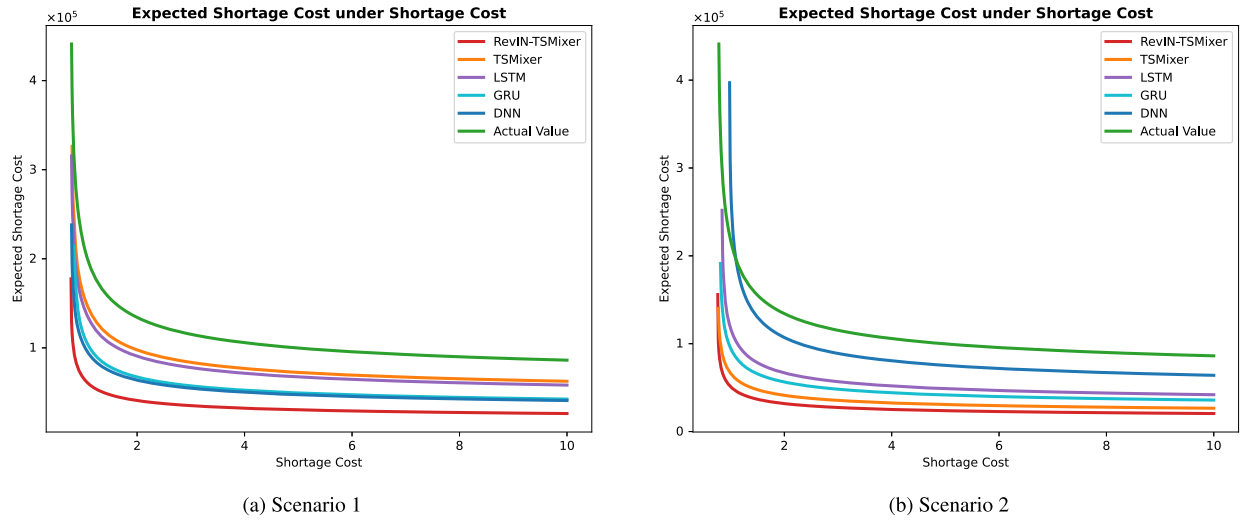


Fig. 11. Expected shortage cost under varying shortage cost per unit across models.

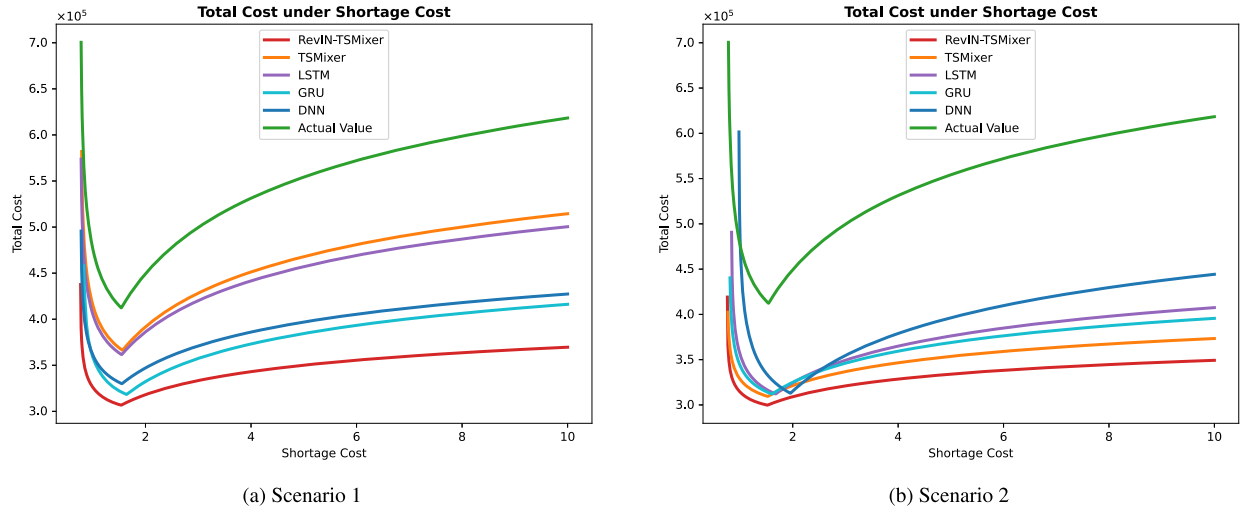


Fig. 12. Total cost under varying shortage cost per unit across models.

a manufacturer in Vietnam, limiting the generalizability of the findings to other industries or international contexts. Second, despite considering shortage costs as a significant variable, the study assumes fixed inventory holding and ordering costs, which may not fully reflect real-world dynamics, as these factors often fluctuate significantly. Lastly, our forecasting model relies solely on macroeconomic indicators and internal variables to forecast demand without considering market factors that could suddenly emerge, such as policy changes, supply chain disruptions, or unexpected events like pandemics; and operational data like inventory history, uncertain delivery time, delayed shipments, etc.

### 5.3. Directions for future research

To address these limitations, future research could pursue several specific directions. Firstly, the model would be extended to accommodate multiple products and applied across various industries for comparative analysis. Secondly, future research should explore the development of more adaptive cost models by incorporating dynamic forecasting techniques that integrate operational data—such as inventory levels, order frequency, and lead times—as predictive features. This integration would allow models to better capture real-time fluctuations in inventory holding costs, ordering behavior, and supply responsiveness, thereby enhancing both accuracy and practical relevance. Thirdly,

future studies could explore integrating advanced forecasting methods, such as reinforcement learning or hybrid models, to enhance adaptability to unusual demand variations, supply-demand shocks, or significant disruptions in supply chains.

Additionally, assessing the long-term effectiveness of the RevIN-TSMixer model during substantial market volatility (e.g., economic crises, pandemics, or significant policy changes) would provide strong empirical support for the robustness and practical applicability of the proposed methodology.

### 5.4. Managerial implications

This study offers several important managerial insights for supply chain managers. Firstly, adopting the RevIN-TSMixer model that integrates inventory management with demand forecasting allows businesses to better balance costs and customer service levels, emphasizing optimization based on economic efficiency rather than solely forecast accuracy. Secondly, the RevIN normalization technique has demonstrated superior effectiveness in stabilizing and enhancing forecast accuracy under conditions of high volatility and non-linearity; thus, managers should consider adopting similar normalization techniques in their forecasting processes. Finally, this research highlights the importance of closely integrating forecasting models with inventory management strategies, enabling firms to proactively develop flexible strate-

gies, optimize resources, and enhance competitive advantage in today's highly dynamic business environment.

## 6. Conclusion

This study effectively accomplished its main goals and made substantial contributions to the comprehension and implementation of sophisticated forecasting demand approaches and optimizing inventory performance. This paper presents a comprehensive analysis of the multivariate time series demand forecasting problem and its intersection with inventory management, providing a strong basis for addressing complex predictive difficulties. The TSMixer model underwent a rigorous analysis, including detailed documentation of its overall structure, individual elements, and the particular design of each layer. This research provided insights into the internal mechanisms of the model and its appropriateness for time series forecasting.

Our proposed RevIN-TSMixer model has demonstrated its effectiveness in cost efficiency. Evaluated under two distinct scenarios—error optimization and cost optimization—RevIN-TSMixer has achieved (1) improved forecasting accuracy compared to the standard TSMixer model, (2) optimized total inventory cost across both scenarios and (3) enhanced inventory performance over comparison models. The reasons are explained below:

- (1) RevIN effectively normalizes input data at both training and inference stages, addressing distributional shifts and improving the model's ability to capture demand patterns. This enhancement leads to more accuracy in forecasting results.
- (2) While comparison models focus solely on improving forecasting accuracy, RevIN-TSMixer balances both forecasting precision and inventory cost minimization. This dual focus enables the model to achieve lower total costs while maintaining inventory efficiency.
- (3) By providing more stable and accurate demand predictions, RevIN-TSMixer helps firms maintain lower safety stock and reorder points while minimizing stockout probability. This contributes to better overall inventory performance compared to other models.

Furthermore, a probabilistic inventory model is employed to evaluate the impact of forecasting accuracy on inventory performance, offering a comprehensive assessment of how different models influence stockout probability, safety stock, reorder point, expected shortage cost, and total cost. From the inventory management perspective, this study indicated several significant insights. First, it highlights the critical role of the shortage cost in inventory performance across forecasting models. The findings suggest that as shortage costs increase, companies tend to maintain higher safety stock and reorder points to mitigate stockout risks, regardless of forecasting accuracy. Second, while forecasting accuracy influences inventory performance, its impact is more pronounced in cost-driven optimization scenarios. In particular, despite the worse forecasting accuracy, the proposed RevIN-TSMixer model demonstrates superior cost efficiency by achieving lower total costs and reducing safety stock compared to other models. Lastly, if the forecasting models prioritize accuracy, the differences in inventory performance across forecasting models become less significant under high shortage costs. This suggests that as the shortage cost increases, inventory strategies converge, reducing the impact of forecasting accuracy on stockout risk. However, when models focus on cost optimization, the difference becomes more pronounced, particularly in terms of total cost. This indicates that cost-driven forecasting approaches have a greater influence on inventory efficiency, reinforcing the importance of selecting an appropriate forecasting model based on the specific optimization objective.

In future work, the application of the TSMixer model in several fields to verify its adaptability and efficacy in diverse forecast complexities will be investigated. Modifications will involve the incorporation of sophisticated deep learning methods, such as attention processes derived from Transformer models, to enhance the ability to capture long-term relationships. Moreover, the efforts will be directed towards expanding

the model to handle bigger datasets and enhancing hyperparameter optimization using AutoML, finally leading to the development of more resilient and flexible forecasting systems.

## CRedit authorship contribution statement

**Thi Ngoc Anh Nguyen:** Conceptualization, Methodology, Data curation, Writing – original draft, Writing – review & editing, Supervision; **Thi Xuan Hoa Nguyen:** Conceptualization, Methodology, Writing – original draft, Writing – review & editing, Supervision; **Ngoc Thang Tran:** Software, Writing – review & editing; **Thi Ha Nguyen:** Software, Visualization, Investigation; **Phuong Anh Nguyen:** Data curation, Writing – original draft, Visualization, Investigation; **Hai Anh Vu:** Data curation, Writing – original draft, Writing – review & editing.

## Data availability

Data will be made available on request.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgement

This work was funded by Rikkeisoft corporation and supported by the Institute for Digital Technology and Economy (BK Fintech), Hanoi University of Science and Technology in the project BKFintech-2024.02.

## References

- AbdElkader, A. G., ZainEldin, H., & Saafan, M. M. (2025). Optimizing wind power forecasting with RNN-LSTM models through grid search cross-validation. *Sustainable Computing: Informatics and Systems*, 45, 101054. <https://doi.org/10.1016/j.suscom.2024.101054>
- Babai, M. Z., Dai, Y., Li, Q., Syntetos, A., & Wang, X. (2022). Forecasting of lead-time demand variance: Implications for safety stock calculations. *European Journal of Operational Research*, 296(3), 846–861. <https://doi.org/10.1016/j.ejor.2021.04.017>
- Barros, J., Cortez, P., & Carvalho, M. S. (2021). A systematic literature review about dimensioning safety stock under uncertainties and risks in the procurement process. *Operations Research Perspectives*, 8, 100192. <https://doi.org/10.1016/j.orp.2021.100192>
- Belete, D. M., & Huchaiah, M. D. (2022). Grid search in hyperparameter optimization of machine learning models for prediction of HIV/AIDS test results. *International Journal of Computers and Applications*, 44(9), 875–886. <https://doi.org/10.1080/1206212X.2021.1974663>
- Cui, L., Chen, Y., Deng, J., & Han, Z. (2024). A novel attLSTM framework combining the attention mechanism and bidirectional LSTM for demand forecasting. *Expert Systems with Applications*, 254, 124409. <https://doi.org/10.1016/j.eswa.2023.124409>
- Demizu, T., Fukazawa, Y., & Morita, H. (2023). Inventory management of new products in retailers using model-based deep reinforcement learning. *Expert Systems with Applications*, 229, 120256. <https://doi.org/10.1016/j.eswa.2023.120256>
- Douaioui, K., Oucheikh, R., Benmoussa, O., & Mabrouki, C. (2024). Machine learning and deep learning models for demand forecasting in supply chain management: A critical review. *Applied System Innovation (ASI)*, 7(5). <https://doi.org/10.3390/asi7050093>
- Gneiting, T. (2011). Making and evaluating point forecasts. *Journal of the American Statistical Association*, 106(494), 746–762. <https://doi.org/10.1198/jasa.2011.r10138>
- Goltos, T. E., Syntetos, A. A., Glock, C. H., & Ioannou, G. (2022). Inventory-forecasting: Mind the gap. *European Journal of Operational Research*, 299(2), 397–419. <https://doi.org/10.1016/j.ejor.2021.07.040>
- Hasni, M., Babai, M. Z., & and, B. R. T. (2024). A hybrid LSTM method for forecasting demands of medical items in humanitarian operations. *International Journal of Production Research*, 62(17), 6046–6063. <https://doi.org/10.1080/00207543.2024.2306904>
- Kourntzes, N., Trapero, J. R., & Barrow, D. K. (2020). Optimising forecasting models for inventory planning. *International Journal of Production Economics*, 225, 107597. <https://doi.org/10.1016/j.jpe.2019.107597>
- Kück, M., & Freitag, M. (2021). Forecasting of customer demands for production planning by local k-nearest neighbor models. *International Journal of Production Economics*, 231, 107837. <https://doi.org/10.1016/j.jpe.2020.107837>
- Liu, Y., Young, R., & Jafarpour, B. (2023). Long-short-term memory encoder-decoder with regularized hidden dynamics for fault detection in industrial processes. *Journal of Process Control*, 124, 166–178. <https://doi.org/10.1016/j.jprocont.2023.01.015>
- Ma, S. (2024). Retail store-SKU level replenishment planning with attribute-space graph recurrent neural networks. *Expert Systems with Applications*, 249, 123727. <https://doi.org/10.1016/j.eswa.2024.123727>

- Manatkar, R. P., Karthik, K., Kumar, S. K., & Tiwari, M. K. (2016). An integrated inventory optimization model for facility location-allocation problem. *International Journal of Production Research*, 54(12), 3640–3658. <https://doi.org/10.1080/00207543.2015.1120903>
- Owais, M. (2025). How to incorporate machine learning and microsimulation tools in travel demand forecasting in multi-modal networks. *Expert Systems with Applications*, 262, 125563. <https://doi.org/10.1016/j.eswa.2024.125563>
- Riachy, C., He, M., Joneidy, S., Qin, S., Payne, T., Boulton, G., Occhipinti, A., & Angione, C. (2025). Enhancing deep learning for demand forecasting to address large data gaps. *Expert Systems with Applications*, 268, 126200. <https://doi.org/10.1016/j.eswa.2024.126200>
- Salas-Navarro, K., Romero-Montes, J. M., Acevedo-Chedid, J., Ospina-Mateus, H., Florez, W. F., & Cárdenas-Barrón, L. E. (2023). Vendor managed inventory system considering deteriorating items and probabilistic demand for a three-layer supply chain. *Expert Systems with Applications*, 218, 119608. <https://doi.org/10.1016/j.eswa.2023.119608>
- Seyedan, M., Mafakheri, F., & Wang, C. (2023). Order-up-to-level inventory optimization model using time-series demand forecasting with ensemble deep learning. *Supply Chain Analytics*, 3, 100024. <https://doi.org/10.1016/j.sca.2023.100024>
- Sukolkrit, N., Arunyanart, S., & Apichottanakul, A. (2024). An open innovative inventory management based demand forecasting approach for the steel industry. *Journal of Open Innovation: Technology, Market, and Complexity*, (p. 100407). <https://doi.org/10.1016/j.joitmc.2024.100407>
- Syntetos, A. A., Boylan, J. E., & Disney, S. M. (2009). Forecasting for inventory planning: A 50-year review. *Journal of the Operational Research Society*, 60(sup1), S149–S160. <https://doi.org/10.1057/jors.2008.173>
- Tliche, Y., Taghipour, A., & Canel-Depitre, B. (2020). An improved forecasting approach to reduce inventory levels in decentralized supply chains. *European Journal of Operational Research*, 287(2), 511–527. <https://doi.org/10.1016/j.ejor.2020.04.044>
- Tratar, L. F. (2010). Joint optimisation of demand forecasting and stock control parameters. *International Journal of Production Economics*, 127(1), 173–179. <https://doi.org/10.1016/j.ijpe.2010.05.009>
- van der Haar, J. F., Wellens, A. P., Boute, R. N., & Basten, R. J. I. (2024). Supervised learning for integrated forecasting and inventory control. *European Journal of Operational Research*, 319(2), 573–586. <https://doi.org/10.1016/j.ejor.2024.07.004>
- Wang, S., Kang, Y., & Petropoulos, F. (2024). Combining probabilistic forecasts of intermittent demand. *European Journal of Operational Research*, 315(3), 1038–1048. <https://doi.org/10.1016/j.ejor.2024.01.032>
- Wellens, A. P., Boute, R. N., & Udenio, M. (2024). Simplifying tree-based methods for retail sales forecasting with explanatory variables. *European Journal of Operational Research*, 314(2), 523–539. <https://doi.org/10.1016/j.ejor.2023.10.039>
- Winston, W. L. (2022). *Operations research: Applications and algorithms*. Cengage Learning. <https://books.google.com.vn/books?id=Y9NYEAAQBAJ>
- Wu, H., Zhou, H., Long, M., & Wang, J. (2023). Interpretable weather forecasting for worldwide stations with a unified deep model. *Nature Machine Intelligence*, 5(6), 602–611. <https://doi.org/10.1038/s42256-023-00667-9>
- Wu, S., Wang, J., Xu, H., Zhao, S., & Xu, J. (2024). Knowledge-based bi-correction model for achieving effective lag-free characteristic on daily urban water demand forecasting. *Expert Systems with Applications*, 255, 124508. <https://doi.org/10.1016/j.eswa.2024.124508>
- Zhang, C., Tian, Y. X., & Hu, A. Y. (2025). Utilizing textual data from online reviews for daily tourism demand forecasting: A deep learning approach leveraging word embedding techniques. *Expert Systems with Applications*, 260, 125439. <https://doi.org/10.1016/j.eswa.2024.125439>
- Zhang, L., & Jánošík, D. (2024). Enhanced short-term load forecasting with hybrid machine learning models: Catboost and XGBoost approaches. *Expert Systems with Applications*, 241, 122686. <https://doi.org/10.1016/j.eswa.2023.122686>
- Zhuang, X., Yu, Y., & Chen, A. (2022). A combined forecasting method for intermittent demand using the automotive aftermarket data. *Data Science and Management*, 5(2), 43–56. <https://doi.org/10.1016/j.dsm.2022.04.001>
- Zuo-Jun Max Shen, L. V. S. (2019). *Fundamentals of supply chain theory*. John Wiley & Sons, Ltd.